Prompts to Production

Comparing Prompt-Driven and Specification-Driven AI Software Development — Behavioural Drift, Integration Reliability, and Regression in a Deterministic Grid-World Controller Benchmark

Gideon Zeru

BSc Computer Science @UCL

Submission date: 24 April 2026

Supervisors: Professor Graham Roberts and Professor Lee Scott

Abstract

Large language models can generate non-trivial code, but the more important engineering question is whether AI-generated code remains correct and stable across iterative revisions. This dissertation compares Prompt-Driven Development (PDD) and Specification-Driven Development (SDD) in a deterministic grid-world robotics benchmark, using controllers generated by GPT-5.4 and Claude Opus 4.7. Two findings motivate the study: SDD improves structural reliability but does not reduce behavioural drift; and drift must be decomposed into regression, improvement, and pure behavioural change to be informative, rather than aggregated into a single count.

The artefact contribution is a deterministic evaluation harness: a simulator, a 15-scenario suite, a controller-loading and trace-capture pipeline, drift classification, and an automated test suite. The central argument is that AI-assisted development must be treated as an engineering discipline, and that behavioural drift must be measured directly on versioned artefacts rather than inferred from demonstrations.

The dataset comprises 107 controllers evaluated across 15 deterministic scenarios, yielding 1605 runs and 1215 adjacent-version pairwise comparisons. Analysis focuses on twelve core lineages (three per model-workflow cell; see §6.2 for selection rationale). SDD improves integration validity and peak performance in both model sources, but does not suppress drift: GPT-5.4 SDD achieves a drift rate of 0.95 versus 0.35 for GPT-5.4 PDD, while Opus 4.7 SDD reaches 0.67 versus 0.72 for Opus 4.7 PDD. Notably, gpt_sdd_a version 3 exactly matches the hand-written oracle on both outcome and step count across all 15 scenarios, while opus_sdd_a version 1 solves 9/15, the highest non-target score in the repository. Yet both lineages show high adjacent-version regression risk, illustrating that specification discipline and behavioural stability are distinct properties.

The qualified answer to the research question is that specification discipline increases structural reliability and peak performance, but transforms rather than eliminates drift. For GPT-5.4, SDD enables oracle-equivalent behaviour on one lineage while introducing very high drift rates. For Opus 4.7, SDD produces the strongest non-target controller in the repository while drift and regression persist. The main conclusion for practice is that advancing AI-assisted software development beyond informal prompting requires explicit contracts, deterministic evaluation, trace-level measurement, and active regression guards—not simply longer or more detailed prompts.

Contents

1. Introduction

AI-assisted programming frequently seems to be a method for increasing efficiency: You provide an input prompt and out pops the program, which results in faster development times. This perspective focuses on a very accurate strength of large language models; however it misses the core issue from an engineering standpoint. Very little software programming happens only once. Programs go through a constant cycle of revisions, integration with other parts of the system, and the expectation that the new code still retains its functionality despite having new demands placed upon it.

This current research looks at the issue in an experimental and controlled environment. Here, the artifact is a grid world controller where the robot navigates through an obstacle-laden environment from the starting point to the destination point, and it does not involve programming once, but rather involves rewriting over iterations following different prompts. Thus, in a way, the experiment is the development process assisted by AI itself.

There are two dimensions to this comparison. The first is discipline of workflow. With Prompt-Driven Development, the underlying model is driven primarily by task specifications, evaluation results, and prompts for refining the current controller. Specification-Driven Development achieves this same high-level objective via a specific contract regarding file structure, observation system, action space, heading convention, progress reasoning, planning constraints, failure handling, and outputs. The second dimension is that of model origin. The model repository includes several controller families developed by both GPT-5.4 and Claude Opus 4.7, which allows for the additional question of whether there is a difference between PDD and SDD when used with different AI models.

Behavioral drift is the main theme of the entire dissertation. By behavioral drift, we mean any detectable difference in behavior between two consecutive versions of the controller within a stable environment. The new version may achieve the same objective, but through a new approach, by adding extra steps, timing out in a situation it had managed before, or by modifying the way it handles interruptions. In common prompt-based development, any deviation is easily overlooked because of the casual evaluation process and the noisy nature of the environment.

The principle of determinism cannot be taken for granted in this case since it is essential for methodological purposes. Indeed, should the environment be stochastic, changes from one version to another would be attributed to random actions of the simulator rather than the code that was produced by means of artificial intelligence. Keeping the simulator constant allows studying the desired effect.

The framing further avoids an initial misunderstanding of the concept. The work is not concerned with whether one algorithm can perform a simple navigation task. Rather, it seeks to drive AI-assisted software development towards a more disciplined approach to engineering. The importance of the benchmark lies in its ability to measure controller performance, reveal interface errors, and detect regressions. This is why the system comprises more than just the controller; it includes scenarios, controller loading, tracing, CSV output, drift reporting, and testing.

Organization of this report is as follows. Chapter 2 places the present study in the context of AI code generation, prompt sensitivity, requirements engineering and empirical software engineering. In Chapter 3, the research question is stated and the final scope is clarified. Chapter 4 describes the artifact and its architecture. In Chapters 5 and 6, the methodology is presented along with the deterministic criteria

ensuring clear distinction between PDD and SDD. Chapter 7 presents results, while Chapters 8 through 11 analyze these results and consider limitations and future work.

1.1 Contributions

The thesis contributes three major advancements. The first is a deterministic grid-world controller benchmark test harness that allows AI-assisted iterative code generation to be audited behaviorally rather than solely on an aggregate score basis. The test harness tracks complete execution traces, imposes a minimum observation obligation, and classifies intergenerational change in adjacent versions as identical, pure drift, degradation, or improvement. The second contribution is experimental results involving 107 controllers and 1605 deterministic executions (subsets compared in the twelve replicates of the core lineages), demonstrating that specification rigor alone cannot mitigate behavioral drift. In the GPT-5.4 dataset, it produces significant drift while at the same time achieving equivalent oracle behavior in one lineage, whereas in the Opus 4.7 dataset, it produces the strongest non-targeted controller in the dataset while still experiencing significant drift and regression between iterations. Finally, an approach based on the premise that drift needs to be analyzed by decomposition, since simply calculating drift obscures whether drift improves or diminishes functionality or changes in some exploratory fashion. Taken together, these results point to the conclusion that taking AI-assisted software development to the next level requires more contracts, determinism, and analysis, not just bigger models or longer prompts.

2. Background And Related Work

2.1 AI-assisted coding as an engineering problem

Large language models have exhibited excellent code generation capabilities through benchmark tests and coding assessments. Chen et al. developed Codex, proving that language models capable of training on code had the capacity to solve a significant number of functional programming problems using HumanEval (Chen et al., 2021). The development of the Mostly Basic Programming Problems dataset was made by Austin et al., where the scaling of synthesis capabilities was shown to be log-linear with respect to model size and natural language feedback was found to almost halve the error of model output predictions (Austin et al., 2021). Subsequently developed benchmarks include more realistic scenarios related to software engineering. For example, the SWE-bench benchmark poses code generation as an issue resolution process at the level of repositories with multiple files, where modern models solve less than 0.5% of real GitHub issues (Jimenez et al., 2024). Another example is the LiveCodeBench benchmark, which implements continuous updates and contamination prevention mechanisms and demonstrates that achieving success on the HumanEval benchmark does not guarantee good results for other programming tasks (Jain et al., 2024). Moreover, Liu et al. find that tests used in the HumanEval benchmark and its analogs are not enough to detect many errors in the generated code, and adding tests improves model performance by 19.3–28.9 percent points (Liu et al., 2023). It should be noted that after the publication of the Codex paper, the discussion of AI code generation became more focused on the concepts of productivity, acceleration, and assistance.

However, generating usable code does not equate to sound software engineering practices. Vaithilingam, Zhang, and Glassman showed that although programmers saw definite benefits from AI-based programming, they had issues with how well the expectations matched actual implementation practices,

such as the necessity of rigorous checking and difficulties in comprehending the behavior of the models used (Vaithilingam et al., 2022). This is important in this context, as even a seemingly sensible solution may prove inadequate because it fails to match a local interface or introduces unsupported states or extensive rewriting on each pass through revision.

This fear exists within the context of an even greater history of software engineering. The classic theory of software evolution by Lehman states that a piece of software, if left alone in a real world environment, will become less and less desirable over time and that its complexity will increase iteratively if not actively reduced (Lehman, 1980). Sculley et al. present another observation relevant to machine learning, which states that machine learning programs accrue technical debt not in the code base but in the system architecture due to boundary erosion, undeclared consumers, and configuration drift that is hard to detect using normal code reviews (Sculley et al., 2015). In turn, artificial intelligence-supported code development adds yet another aspect to both issues because the code is rewritten, not manually altered, and thus its boundaries and the maintenance burden are dynamically altered by the underlying code generation process.

Fan et al. study this nascent field holistically and classify the use of LLMs in programming, design, specification, debugging, refactoring, and documentation, to determine that the integration of conventional software engineering approaches with the unique capabilities of LLMs is what holds the key for obtaining dependable results (Fan et al., 2023). In this respect, this research adopts an approach that considers AI-enabled software development from a perspective of software engineering process. The focus is not only on whether "the model can generate code" but also "what engineering behavior will a certain prompting discipline generate?"

2.2 What separates PDD from SDD

This difference is more than the mere fact that one prompt is shorter while another is longer. The difference is in how the definition of the task is provided.

With the PDD approach, the definition of the task is left implicit. While receiving the objective, hints to the interface and feedback from earlier executions, the model should figure out much of its surroundings on its own. With the SDD approach, the definition of the task is externalized. All requirements that might otherwise have been latent need to be put in writing: the existence of the controller class, the allowed observation fields, the fixed action and heading enums, the inference rules for blocked movement, and what kind of state and recovery mechanisms are permitted.

This is quite consistent with the motivation for requirements engineering overall. As per the ISO/IEC/IEEE 29148 standard, requirements engineering can be defined as the systematic generation and management of information artifacts describing what a system must achieve and how (ISO/IEC/IEEE, 2018). Similarly, the SDD prompts proposed in the dissertation are not a specification language but have the same engineering sensibility in terms of making assumptions clear and not leaving anything to inference by the generator.

2.3 Why prompt structure changes model behaviour

An explanation of the distinctions between PDD and SDD must also include a brief description of the nature of these two types of artificial intelligence. A language model is not a compiler for natural

languages. It is a statistical next-token predictor which has been trained on enormous amounts of text and code. With any input, it predicts an output sequence that is likely within the context it has learned.

In case of an incomplete prompt, the model uses the knowledge that it gained from training to fill the missing information. This can be completely valid in a general sense yet incorrect when considering the specific project in question. An example is the assumption about the presence of an obstacle sensor based on the frequency of occurrence of similar code in robotics projects. Another thing is that the model may do more than it was asked, since there may be multiple ways to achieve a certain result.

The research in prompt sensitivity also emphasizes this fact. In the paper introducing ProSA, Zhuo et al. demonstrate that the model's performance depends significantly on the prompt selection and prompt design and is not invariant with respect to the instruction meaning (Zhuo et al., 2024). It is evident from the studies in prompt engineering that using structured prompts to reveal model reasoning or impose constraints influences the tasks that the model can perform successfully: Wei et al. demonstrate that providing the examples of chain-of-thought solutions results in a significant improvement in the model's performance in arithmetical, commonsense, and symbolic reasoning tasks (Wei et al., 2022), thereby substantiating the general idea that prompt structuring affects not only visual but also functional model behavior. From the perspective of the coding generation setting, this issue acquires great relevance. Namely, if prompt structuring defines the task, then changes in the model behavior across versions may occur due to changes in the prompt itself rather than other factors.

Why is SDD important? It reduces the problem of inference. Rather than relying on the model to figure out the file contract, observation interface, recovery rules, and memory constraints, the prompt simply specifies them. This does not solve the problem of variance. It transforms it. In an SDD approach, the model might do a massive rewrite, but it is rewriting within a clearer set of constraints.

2.4 Determinism and empirical credibility

The purposeful elimination of randomness is another attribute of this benchmark setting. For every scenario, there will be a definite geometry, start-goal, heading convention, and step allowance. Additionally, the random algorithm’s seed will be known beforehand, allowing it to be reproducible. This ensures that only one particular execution sequence per controller-scenario pairing exists.

The approach aligns with the empirical software engineering principle that, if the research subject is unstable, then the measuring context must be stable. The focus on control and explicit design, along with the relationship between validity of measurement and credibility of inferences drawn from the experiment, feature prominently in Wohlin's treatment of experimentation in software engineering (Wohlin et al., 2024). In the present dissertation, however, the unstable object of study is the generated controller lineage rather than the simulator itself. Deterministic methods lend greater strength to the inference. Such methods are particularly salient when taking into consideration the additional list of threats to validity that Sallou, Durieux and Panichella identify in their discussion of LLM-powered software-engineering research (Sallou et al., 2024). These include closed-source updates to models, contamination between training datasets and test benchmarks, and non-reproducible outputs over time. Deterministic method, in concert with fixed observation contracts and archived prompt artefacts, addresses many of these concerns through making the dependent variable into a tangible code artefact.

2.5 Robotics context and specification discipline

While the standard is theoretical, this does not make the issue an easy one due to the nature of the theoretical construct. Self-directed and robotic systems are precisely the type of programs that require careful consideration and analysis when it comes to interface discipline, behavior evaluation, and trustworthy validation. Luckcuck et al. state that testing and simulating can be inadequate means of ensuring strong assurance in autonomous robots and emphasize the importance of specifying and verification-oriented thinking (Luckcuck et al., 2020). While the current project does not make claims towards formal verification, it uses that approach to make the behavior accountable.

Two distinctions about the term "specification" should be stated up front to avoid overclaiming. First, the SDD prompts used in this dissertation are structured natural-language contracts; they are not formal specifications in the sense of TLA+, Alloy, or refinement calculi. They have no machine-checkable semantics, and compliance is verified empirically through behaviour on the benchmark rather than through proof obligations. Second, SDD here refers to a prompt-authoring discipline, not to a tool-driven workflow such as GitHub Spec Kit, which wraps specification authoring inside a multi-stage pipeline with its own scaffolding (project constitution, planning, task creation, implementation). The distinction matters because it narrows the claim: where this dissertation says "specification discipline improves X", the cause under test is the explicit-contract prompt itself, not any surrounding orchestration tooling. §2.6 discusses why Spec Kit was deliberately not adopted for this reason.

2.6 Why Spec Kit is relevant, but was not used

The Spec Kit offered by GitHub can be considered relevant to the current dissertation because it embodies a larger trend of moving towards a specification-based AI model. The process involved in its operation includes a number of stages based on the following sequence: project constitution, specification, planning, task creation, and implementation (GitHub, 2026), unlike being focused on one prompt only.

However, the Spec Kit wasn't applied in the experiment, and this decision was quite reasonable. Had a full suite of tools been incorporated with its own scaffolding, clarification processes, and created artifacts, it would have made the experiment incapable of isolating the impact of the specification discipline per se because improvements could have been due to the presence of the whole chain of tools and not just because of the transition from prompting to contract specification. By staying at the level of the prompt, the experiment manipulates only the discipline of the prompt itself.

In a larger sense, this research effort itself exists within the context of a shift within AI-augmented programming practices. The initial stages of programming model usage tend to be centered around a craftlike prompting process. Later stage tooling efforts, such as Spec Kit, focus on artefactualization, clarification processes, and constraints. This dissertation adds to the shift by providing rationale for its importance. After controllers have been analyzed within a deterministic harness, the cost of implicitness becomes clear through interface failure, stagnation, or regression-focused rewrite cycles.

2.7 Assessment criteria for the PDD vs SDD comparison

In order to ensure concreteness and repeatability in the comparison, the criteria for assessment are determined before the presentation of the results in Chapter 7. The criterion is defined using a measure which can be calculated objectively on the basis of the artefacts described in §4.8.

- Integration validity — does the generated controller load and execute under the minimal observation contract? Measured by the rate of INVALID_CONTROLLER events on the first version of each lineage.
- Success rate — how many of the 15 scenarios end in GOAL_REACHED for a given version? Reported at both best-version and final-version granularity.
- Behavioural equivalence to oracle — does the generated controller produce the same (event, step_count) tuple as the hand-written oracle on every scenario? This is a strictly stronger criterion than matching solved-scenario count.
- Adjacent-version drift — of the 15 scenarios between two adjacent versions, how many show any observable change in behaviour? Classified per-scenario into same / drift / regression / improvement (§5.8).
- Regression risk — what fraction of adjacent-version comparisons strictly reduce the success count? High regression risk indicates that revision under a given workflow is unreliable.
- Improvement efficiency — when a revision improves the success count, by how many scenarios on average? This distinguishes exploratory drift from directed progress.

These six criteria serve to operationalize the four research questions outlined in §3.1 and become the interpretive framework of Chapter 7. Whenever a number appears in the body text, it will be compared to one of these six criteria, and that criterion will be mentioned.

3. Research Question, Aims, And Scope

3.1 Research question

The central research question is:

Does Specification-Driven Development reduce behavioural drift and structural unreliability relative to Prompt-Driven Development during iterative AI-assisted controller development in a deterministic grid-world benchmark?

That question naturally breaks into four sub-questions:

1. Does SDD improve interface and integration reliability?
2. Does SDD improve behavioural performance, measured through success, timeout, and efficiency?
3. Does SDD reduce version-to-version drift, or does it instead enable larger but riskier rewrites?
4. Do the answers depend on model source, specifically GPT-5.4 versus Claude Opus 4.7?

3.2 Hypotheses

These sub-questions translate into four working hypotheses that the benchmark is designed to test. H1: under a minimal observation contract, SDD will produce fewer integration-level failures (INVALID_CONTROLLER events on first generation) than PDD. H2: SDD will achieve equal or greater best-version success rate than PDD within the same model source. H3: SDD will not monotonically reduce version-to-version behavioural drift; instead, it may enable larger rewrites that introduce per-scenario regression as well as improvement. H4: the effect of specification discipline is model-source-

dependent, and the PDD-vs-SDD ordering observed within the GPT-5.4 corpus will not necessarily replicate within the Opus 4.7 corpus. The hypotheses are exploratory rather than pre-registered: they are falsifiable against the data collected, but the small per-cell lineage count (n = 3) limits the strength of any rejection or confirmation claim, as discussed in §9.1.

3.3 Aims

The final project has three practical aims.

1. Build a deterministic controller-evaluation framework in which AI-generated revisions can be compared fairly.
2. Use that framework to compare PDD and SDD across repeated controller lineages.
3. Interpret the findings as software engineering evidence about drift, regressions, integration reliability, and maintainability rather than as isolated benchmark scores.

3.4 Scope refinement

It seems as though there were expectations for a larger scope of study at the outset, including additional software environments and an evaluation suite. The product produced is more specific, but more rigorous from a methodology perspective. This is achieved by studying only a single environment where deterministic controllers exist; thus, more rigorous attribution is possible.

One scope boundary deserves explicit statement so that the findings below are not read as broader than they are. Every quantitative claim in this dissertation is made about controllers generated and evaluated inside the deterministic grid-world benchmark described in Chapter 4. The benchmark is a deliberately simplified abstraction: discrete state space, fixed scenario geometry, a minimal observation contract, and a fixed step budget per run. Findings about behavioural drift, oracle equivalence, and workflow-by-model interaction are properties of controllers in this environment. Generalisation to richer robotic settings, to real code repositories, or to production software is plausible but not established by the experiments here; §10.3 discusses the port of the drift-decomposition framework to repository-level benchmarks such as SWE-bench as the natural next step.

3.5 Requirements (MoSCoW)

The obligations of the benchmarking framework can be summarized in the form of the MoSCoW prioritization process, which is outlined below. These represent engineering obligations prior to implementation, and they provide a reference list against which the list of accomplishments, outlined in Chapter 11, will be evaluated. “MUST” obligations represent irrevocable commitments defining the benchmark artefact; “SHOULD” obligations provide empirical value; while “COULD” obligations were out-sourced in cases where time or data did not allow their fulfillment.

ID	Requirement	Priority	Delivered in
R1	Deterministic simulator: fixed geometry, fixed start/goal, fixed step budget, no randomness in environment dynamics.	**MUST**	§4.3

R2	Minimal observation contract (position, heading, goal) enforced for every controller, with integration validity checked before any scenario is run.	**MUST**	§4.2, §7.9
R3	Scenario suite covering easy, medium and hard difficulty tiers with five named challenge types (straight line, single detour, corridor, dead end, counterintuitive).	**MUST**	§4.4, C.5
R4	Per-run trace capture producing (event, step_count, action_sequence) for every (controller, scenario) pair.	**MUST**	§4.5
R5	Adjacent-version drift classification at per-scenario granularity into same / drift / regression / improvement.	**MUST**	§5.8, §7.6
R6	Two model sources evaluated under both PDD and SDD, with deterministic rules separating the workflows (§5.3) so the variable under test is prompting discipline, not prompt length.	**MUST**	§5
R7	Independent oracle, random baseline, and target reference controllers providing lower bound, upper bound, and oracle check.	**MUST**	§4.6
R8	Automated test suite covering smoke execution, drift classification, harness behaviour, and analysis output (15 passing tests from pytest tests -q).	**MUST**	§6, C.6
R9	Reproducibility: scripted reconstruction of every table from the final results.csv, commit-pinned.	**MUST**	C.6
R10	Scale: at least 50 versioned controllers and at least 500 recorded executions, to give meaningful adjacent-version statistics.	SHOULD	§6.1 (107 / 1605 delivered)
R11	Repeated SDD lineages a, b, c for both model sources, to expose between-lineage variance under identical prompts.	SHOULD	§6.2
R12	Integration-failure evidence captured distinctly from behavioural failure, to expose old-spec regression as a repeatable failure mode.	SHOULD	§7.9
R13	Chronological documentation of generation order, so that later analysis cannot silently benefit from knowing model behaviour.	SHOULD	§5.2
R14	Real-time collaboration of multiple AI agents on the same controller revision.	COULD	Descoped — §10.1
R15	Per-step interactive debugger UI for inspecting traces visually.	COULD	Descoped — §10.2
R16	Statistical significance testing across lineages.	COULD	Descoped — §9.1, §10.3

All MUST requirements (R1-R9) have been satisfied. All of R10-R13 have also been satisfied; R10 has even been exceeded substantially (107 controllers instead of 50; 1605 runs instead of 500). The COULD requirements (R14-R16) have not only been descoped but it turned out that such description was necessary as it would have needed different engineering and not extra analysis, considering that the empirical story was already sufficient given the current scale. See §11.1 for detailed account of successes corresponding to the aims presented in §3.2.

4. Artefact And System Design

4.1 Repository overview

It is not only a library containing all the controllers produced by the tool. It also represents an empirical testing environment for the developed software. The package gridbot.sim provides definitions of action types, heading types, events, state updates, and traces creation. The package gridbot.world is responsible for the grid abstraction and the definition of the observations that will be given to controllers. Finally, the package gridbot.eval is responsible for scenarios, test sets, controller loading, records creation, serialization of results, and drift computation. The main directory contains experiment runner and summary CSV files.

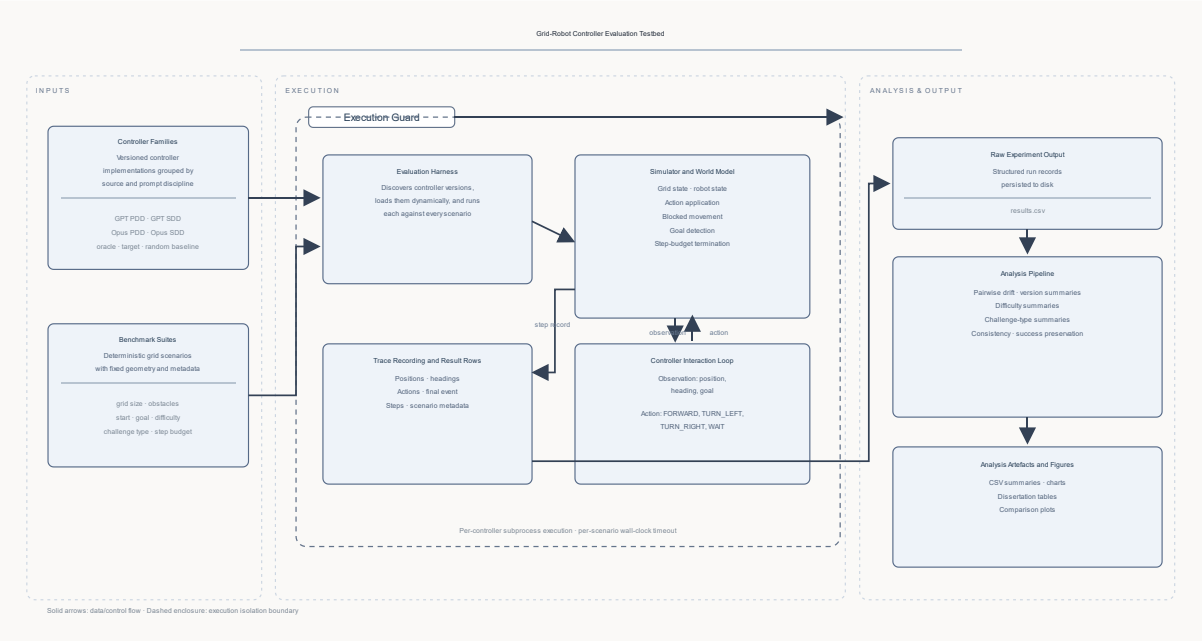

Figure 1. Evaluation testbed architecture for deterministic controller execution, trace capture, result serialisation, and downstream drift analysis.

This architecture matters because it supports a stronger claim than "I used an LLM to write some code." The project built the machinery needed to inspect, compare, and criticise AI-generated software artefacts.

4.2 Controller interface

The controller contract is intentionally minimal. Each controller must define:

```
class Controller:
```

```
def act(self, observation) -> Action:
    ...
```

Fields that are guaranteed to be present in the final benchmark include observation.position, observation.heading, and observation.goal. Actions the controller can produce include Action.FORWARD, Action.TURN_LEFT, Action.TURN_RIGHT, and Action.WAIT.

In terms of the minimal contract, this represents one of the best design decisions made with respect to the artefact, since it reveals the distinction between reasonable code and working code. Many of the controllers that are generated appear reasonable but fail because they lack Controller functionality, while others fail due to reliance on undefined fields like observation.front_blocked.

4.3 Deterministic simulator and collision semantics

The simulation begins at the same location and initial orientation, which is east. Each action affects the current state predictably. Rotation alters the direction, waiting uses up one unit of time, and moving ahead advances the robot when the move is unblocked and otherwise leaves it in place.

One methodological aspect worth noting here is as follows. At an earlier point of the process of development, collision was regarded as a fatal failure. In the last simulator, however, blocked progression is not considered fatal either; it costs a move and eventually results in TIMEOUT once the number of moves becomes insufficient. This alteration was made so that the traces would become more meaningful. The initial approach based on fatal immediate collisions generated too many short runs which could not be used to detect drifts.

However, this does not obscure failure. Rather, it transforms the failure indicator from a termination to one of bounded non-progress. But this comes at the price of the resulting data set registering a great many collisions as timeouts.

For complete clarity, the terminal-event enum reported in results.csv contains four values: GOAL_REACHED, TIMEOUT, COLLISION, and INVALID_CONTROLLER. After the non-fatal-movement change above, the COLLISION value is retained in the enum for schema stability and for historical runs made under the earlier fatal-collision behaviour, but no run in the final dataset used for the analyses in Chapters 7 and 8 ends in COLLISION. Failures caused by blocked movement are therefore all represented as TIMEOUT, and any analysis that talks about "collision-like" failures in the main body is referring to the TIMEOUT population. This single definitional rule is relied on throughout the dissertation and avoids the need to reconcile collision and timeout counts at later points in the text.

4.4 Scenario suite

There are 15 scenarios in the benchmark test that are classified according to their difficulty level into easy, medium, and difficult categories. The types of challenges in each scenario include linear paths, single detours, corridors, dead ends, and counter-intuitive routes.

Easy maps are the basis for discrimination. Maps with one detour map test local avoidance. Corridor maps test the commitment to navigate in restricted geometry. Maps with dead ends test the ability of the controller to recover from such situations. Counter-intuitive maps test if the controller can make a temporary retreat from the target if the chosen path is incorrect.

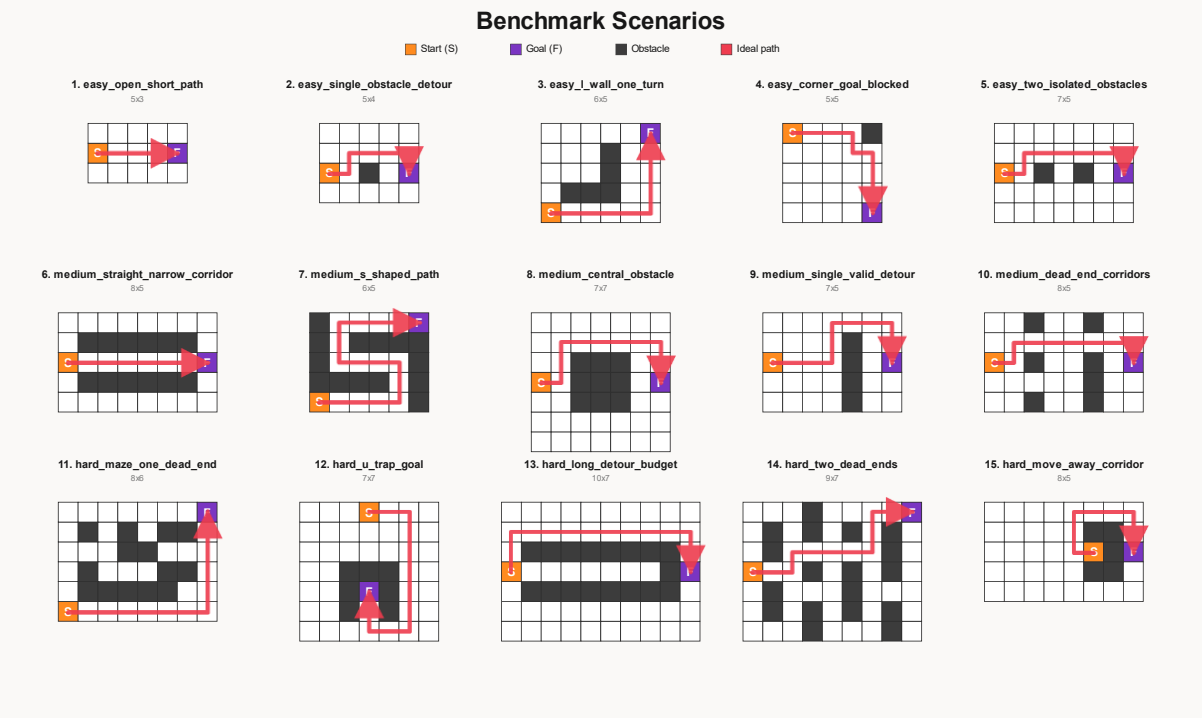

Figure 2. Full deterministic benchmark scenario suite used in the evaluation, showing start and goal cells, obstacle placement, and one ideal path for each map.

The composition was chosen to maximise discriminative power across the skills a navigation controller must combine. Five easy scenarios establish a floor: any controller that fails them fails the minimum bar and the failure is informative without needing nuance. Five medium scenarios test whether a controller can commit to a detour, handle corridor geometry, and recover from a dead end. Five hard scenarios include long-budget detours, counterintuitive routes that require temporary retreat from the goal, and dense obstacle fields. The three tiers together cover the full range of success rates observed in the repository, from 0/15 for the random baseline through 15/15 for the target controller, so the suite resolves controllers on the full spectrum rather than saturating at the top or bottom. The five challenge types were chosen so that no single navigation heuristic (greedy goal-directed, wall-following, aggressive retreat) can solve all 15, which in turn means that drift between versions can be localised to specific competences rather than collapsing to one aggregate.

4.5 Recording and analysis pipeline

The harness captures information about the outcome event, number of steps taken, and detailed logs of actions performed, positions, and headings for each run. This data is then serialized into CSV files and further analyzed to produce multiple summaries: version-performance, differences between adjacent versions, difficulty, challenge type, and consistency summaries.

This is the very core of the dissertation, without which behavioral drift would just be an unquantifiable impression. With it, drift is a tangible object.

4.6 Baselines and reference controllers

The repository contains three especially useful comparison points.

1. random_baseline is a deterministic lower bound.
2. oracle is a hand-written reference controller under the same observation contract.
3. target is an upper bound showing that the benchmark is solvable in principle, but it is not a fair comparison controller because it exploits benchmark-specific knowledge.

The presence of all three makes the final comparison much easier to interpret.

4.7 Evidence of engineering quality

This artefact has automated test cases which include smoke testing, harness behaviour testing, drift categorisation testing, and analysis results testing. There are 15 passing tests in the final workspace. This does not mean that the software is formally verified, but it shows that disciplined engineering practices have been followed.

4.8 Data schema

Four core data sources are referenced in the analysis and appendices. Their key fields are outlined below to enable readers to compare statements in the text with the corresponding repository data.

results.csv — per-run master record (1605 rows):

Column	Type	Meaning
approach	string	Controller lineage identifier (e.g. gpt_sdd_a, opus_pdd_group_b3).
version	integer	Version index within the lineage, 1-based.
scenario_id	integer (1-15)	Scenario identifier; see C.5 for the registry.
event	enum	Terminal event: GOAL_REACHED, TIMEOUT, COLLISION, INVALID_CONTROLLER.
steps	integer	Number of steps taken before the terminal event.
action_trace	string	Complete ordered list of Action enum values executed during the run.
error	string or null	Exception text when event = INVALID_CONTROLLER; null otherwise.

version_summary.csv — per-version performance summary (107 rows):

Column	Type	Meaning
approach	string	Controller lineage identifier.
version	integer	Version index within the lineage.

challenge_type	enum	Challenge slice represented by the row; repo-wide rows use all.
scenarios	integer	Number of scenarios represented by the row.
success_count	integer (0–15)	Number of scenarios completed with event = GOAL_REACHED.
timeout_count	integer (0–15)	Number of scenarios ending in TIMEOUT.
invalid_count	integer (0–15)	Number of scenarios ending in INVALID_CONTROLLER.
average_steps	float	Mean steps across the scenarios represented by the row.

pairwise_changes.csv — per-scenario pairwise drift (1215 rows):

Column	Type	Meaning
approach	string	Lineage identifier shared between the two adjacent versions.
from_version	integer	Predecessor version index.
to_version	integer	Successor version index (from_version + 1).
scenario_id	integer (1–15)	Scenario in which the pairwise comparison is made.
change_type	enum	Per-scenario classification: same / drift / regression / improvement.
event_prev	enum	Terminal event of the predecessor on this scenario.
event_curr	enum	Terminal event of the successor on this scenario.

gridbot/eval/benchmark_suite.py — the 15 scenarios, one Scenario definition each:

Field	Type	Meaning
scenario_id	integer	Primary key used throughout the analysis.
name	string	Descriptive label (e.g. easy_open_short_path).
difficulty	enum	easy / medium / hard.
challenge_type	enum	straight line / single detour / corridor / dead end / counterintuitive.

grid_size	tuple(int,int)	Grid dimensions as (width, height).
start	tuple(int,int)	Start cell.
goal	tuple(int,int)	Goal cell.
step_budget	integer	Maximum steps before TIMEOUT.

Each statement in terms of numbers in the body of the paper may be reproduced by importing these files using pandas and aggregating based on either (approach, version) or (approach, version, scenario_id). This is precisely what the scripts in tools/ do.

5. Methodology

5.1 Overall approach

The dissertation uses a controlled empirical methodology. The environment is held fixed while controller lineages vary along two dimensions: prompting discipline and model source. Because the environment is deterministic, the emphasis is not on statistical averaging over random runs. It is on reproducible comparison of concrete artefacts and adjacent revisions.

5.2 Chronology and generation context

The first deterministic testbed was designed at the close of January 2025. PDD tests started on 15 February 2025. SDD tests commenced on 1 March 2025. Such sequencing must be stated to raise a question of validity since subsequent research could have benefited from enhanced infrastructure and greater researcher proficiency.

The evidence provided by the archive and the confirmation from users reveal two models as sources of the data used in the study; namely, GPT-5.4 and Claude Opus 4.7. The generations were carried out in separate sessions using private browsing tabs to prevent one line of controllers from inheriting chat history from others.

According to vendor-published descriptions, OpenAI positions GPT-5.4 as targeted at professional work and coding workflows, while Anthropic describes Claude Opus 4.7 as a notable advance in software-engineering and long-running coding tasks (OpenAI, 2026; Anthropic, 2026). Neither of these sources can be taken as proof of performance in this repository, but still, they provide a reason for using such

models in research.

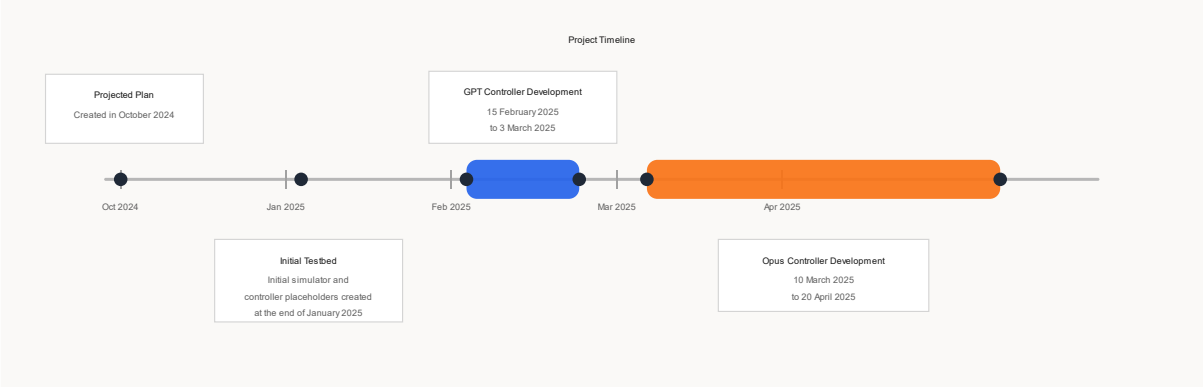

Figure 3. High-level project timeline from initial planning through testbed construction and the GPT and Opus controller-development phases.

One reproducibility caveat about model configuration should be stated explicitly. Controllers were generated through the consumer chat interfaces of GPT-5.4 and Claude Opus 4.7, not through the respective APIs. Sampling parameters such as temperature, top-p, and system prompt settings were therefore not under direct experimental control and are reported as whatever default the interface applied at the time of generation. This means that the exact outputs are not bit-reproducible by re-running the same prompts on the same models, and any replication of the numerical results would need to use the archived controller source code from the repository rather than attempting to regenerate the lineages from the prompts. The benchmark harness itself is fully deterministic: given a fixed controller file, the resulting trace and outcome are reproducible.

5.3 Deterministic rules separating PDD and SDD

The delineation between PDD and SDD was not vague. The approach necessitated deterministic distinctions between prompt types to ensure fairness.

Criteria	PDD requirement	SDD requirement
Model delivery	Same browser-based model interface pattern and isolated sessions	Same browser-based model interface pattern and isolated sessions
Target of output	One controller file implementing the required interface	One controller file implementing the required interface
Task facts sharing	Same simulator, scenario suite, observation contract, action set, and evaluation feedback format	Same simulator, scenario suite, observation contract, action set, and evaluation feedback format
Prompt type	Natural language task setup + run feedback + improvement requests	Explicit numbered contracts for file structure, observations, actions, headings, coordinate system, progress deduction, planning, error handling, and output

Preserving logic	Informal requests such as preserve successful scenarios when feasible	Formal instructions incorporated into writing specifications
Constraint category	Goal-oriented / descriptive	Contract-oriented / prescriptive

This table is important because it shows that the experiment was not simply comparing "short prompts" to "long prompts". It was comparing two different forms of development discipline.

5.4 PDD workflow

The artefacts for the PDD prompt are concise and focused on outcomes. An example is found in controllers/pdd_group_b3/prompt_v3.txt, which starts with the problem statement, includes the import list and observation field, follows with the prior controller outcomes, and concludes with an improvement task for future controllers.

Short excerpt:

```
Write Python code for a grid robot controller...

Previous controller results for this chain:

- Successes: 4/15

...

Improve the controller so it is more successful and more efficient.
```

This style is recognisably prompt driven. It communicates what the developer wants, but not a full behavioural contract for how the model should get there.

5.5 SDD workflow

These are clearly distinguishable through material differences. An instance from the controllers/sdd_a/prompt_v4.txt file can be provided as follows: file contract, observation contract, action contract, heading contract, coordinate convention, inference for blocked progress, behavior requirements, planning constraints, recovery constraints, and output requirements.

Short excerpt:

```
Regenerate the controller and follow this specification exactly.

1. File Contract

...

2. Observation Contract

Use only:

- observation.position

- observation.heading

- observation.goal

...
```

```
6. Progress-Inference Rule

Infer blocked forward movement only when...
```

This is the core methodological difference. The SDD prompt does not simply ask the model to do better. It says what the controller must look like and what assumptions it is allowed to make.

5.6 Why hand-crafted SDD prompts were preferable to Spec Kit here

Spec Kit can be used as a comparative model, but it should not be considered a treatment for the experiment in question. According to GitHub, Spec Kit is a multiphase, specification-driven development approach that involves the development of a constitution, specifications, planning, and tasks generation, not just a prompt template (GitHub, 2026). Had it been applied, the study would have conflated:

1. the effect of explicit specification discipline;

2. the effect of a larger toolkit with its own scaffolding and workflow gates.

The use of artificial prompts to test SDD, on the other hand, will give validity to the comparison because the method of delivery remains unchanged, while the method of prompting changes. The second benefit is that it will increase the intelligibility of the research as the process of Prompted Design Discovery (PDD) and Software Development Discovery (SDD) becomes clear from the prompts created for both. This becomes important due to the fact that the thesis states some relationship with engineering.

The third strength of this type of research is its ability to generalize. Although research conducted with the use of Spec Kit alone can be limited because of the restrictions set by the Spec Kit itself, the research on the basis of prompts created manually allows applying to a wider field of practice of software engineering based on AI. However, what makes the thesis even more credible is the coincidence of GitHub regarding the specification techniques

5.7 Fairness corrections made during development

These two decisions related to design history need to be addressed here, since they are connected to fairness issues.

In the first place, front_blocked was considered and then removed from observation. Keeping front_blocked would have disadvantaged controllers with a more favorable contract. It would also mean that comparison would be harder due to uneven start. By sticking with position, heading, and goal, everything was put on even grounds.

Secondly, failure to move blocked was modified to mean non-fatal failure to advance, which made the trace information more informative, but this did not require randomization. Many times bad controller actions can be seen in timeouts collected.

This is significant, since it shows how this methodology developed into an approach that allowed for a fairer comparison than the one that was more advantageous.

5.8 Units of analysis and metrics

The primary unit of analysis is the controller-version-by-scenario pair. The secondary unit is the adjacent version transition within a lineage.

The main metrics are:

1. `Integration validity`: whether a controller loads and runs under the required interface.

2. `Outcome`: `GOAL_REACHED`, `TIMEOUT`, or `INVALID_CONTROLLER` in the final dataset.

3. `Efficiency`: step count and shortest-path-relative efficiency.

4. `Behavioural drift`: whether adjacent versions are classified as `same`, `drift`, `improvement`, or `regression`.

5. `Pure trace drift`: trace change without outcome or step-count change.

Behavioural drift is central because it captures the hidden cost of iterative AI-assisted development: local requests can produce global behavioural changes.

Using two model sources strengthens that claim rather than distracting from it. If the same PDD and SDD distinction produces different behavioural profiles in GPT-5.4 and Opus 4.7, then the dissertation can say something more precise than "one workflow is always best". It can say that specification discipline changes the development problem, but the response to that change is model contingent. That is a stronger and more realistic engineering conclusion.

6. Experimental Setup

6.1 Dataset scale

The final result file includes 107 versioned controllers across 15 scenarios, which amount to a total of 1605 runs logged. The adjacent version drift analysis includes 1215 per-scenario drift rows analyzed in pairwise_changes.csv and summarized into 81 version-pair drift rows in pairwise_summary.csv. In cases where the drift is reported decomposed by Chapters 7 & 8, such analyses use twelve repeat core lineages, three GPT PDD, three GPT SDD, three Opus PDD, three Opus SDD lineages, as specified in §6.2. Limiting decomposition to the core set ensures that the sample size per cell is consistent (three lineages per workflow cell), and eliminates any potentially misleading groups and approach folders (group_a) not included in any other cell workflow. Whenever an analysis in the results chapter uses all 107 controllers / 1605 runs as opposed to just the core set, this fact is made clear.

6.2 Core families used for the main comparison

The cleanest repeated lineages are:

- `gpt_pdd_group_b1`, `gpt_pdd_group_b2`, `gpt_pdd_group_b3`
- `gpt_sdd_a`, `gpt_sdd_b`, `gpt_sdd_c`
- `opus_pdd_group_b1`, `opus_pdd_group_b2`, `opus_pdd_group_b3`
- `opus_sdd_a`, `opus_sdd_b`, `opus_sdd_c`

The older `group_a` branches are still analytically useful, especially for integration failures and archive context, but the clearest workflow claims come from the `group_b` PDD lines and the repeated `a/b/c` SDD lines.

6.3 Why hard scenarios matter

Hard and extremely difficult tasks are not defects. Rather, they are essential to discrimination. Should there have been only simple problems, then all control systems would seem equally good, and drift would be a matter of concern only on paper. Hard tasks reveal the limits of a program, its flaws, and allow regressive trends to become visible.

6.4 Result sources used in this draft

The draft is mostly based on results.csv along with the summary files generated by results.csv, namely, the chain summary, the challenge-type summary, the difficulty summary, the consistency summary, and the pairwise summary. In case of comparisons that are not covered by the summaries, for example, the opus_* families, the figures used in this draft are obtained from results.csv directly.

7. Results

7.1 High-level picture

The first significant finding is that interface validity is an essential achievement by itself. There are several controllers in the repository that look like sensible interfaces but are unable to form a working system since they miss the necessary Controller class or rely on unavailable fields, like observation.front_blocked. Thus, one can prove one of the basic statements of the dissertation: the artificial intelligence code should be considered a system artifact, not merely plausible text.

The second general discovery is that performance and drift do not correlate linearly. Good controllers need not be stable, and vice versa.

7.2 Core family performance by workflow and model source

Across the clean core families, the aggregate picture is as follows.

Family	Runs	Successes	Success rate	Invalid runs	Timeouts	Mean steps
GPT-5.4 PDD core	360	64	0.178	45	251	17.944
GPT-5.4 SDD core	180	43	0.239	45	92	14.700
Opus 4.7 PDD core	360	56	0.156	15	289	20.139
Opus 4.7 SDD core	180	66	0.367	0	114	18.767

Several conclusions follow immediately.

Firstly, among the GPT-5.4 families, SDD performs better than PDD in terms of success rate and step efficiency. Secondly, among the Opus 4.7 families, SDD again appears to outperform PDD on raw success rate and efficiency, while maintaining structural runnability in the repeatable a/b/c lines. Thirdly, the interaction between prompt discipline and model source is inconsistent. While GPT-SDD emerges as the most accurate family, matching the oracle specifications closely, Opus-SDD produces the highest success counts, despite a dissimilar improvement trajectory.

The table also reveals a more nuanced message concerning structural reliability. The GPT-5.4 SDD family is still incomplete, since it replicates the old-spec "front blocked" bug during its earliest generations. However, the Opus 4.7 SDD family enjoys full structural runnability and behavioural productivity in multiple generations. This observation is significant. In one case, we witness failures due to initial misunderstanding of the model interface and later large-scale rewrites. In the other, we observe superior structural correctness at the outset, followed by difficult behavioural improvements over time.

7.3 Best controllers and comparison to random, oracle, and target

The strongest single versions are especially revealing.

Controller family	Version	Successes	Interpretation
random_baseline	1	0/15	Deterministic lower bound
gpt_pdd_group_b3	3	5/15	Best GPT-5.4 PDD controller
gpt_sdd_a	3	7/15	Best GPT-5.4 SDD controller; exactly matches oracle on event and steps across all 15 scenarios
opus_pdd_group_b3	5	8/15	Best Opus 4.7 PDD controller (peak, not final version)
opus_sdd_a	1	9/15	Best Opus 4.7 SDD controller; strongest non-target controller in the repository
oracle	1	7/15	Hand-written reference under same observation contract
target	2	15/15	Benchmark-specific upper bound; not a fair general controller

This table provides the examiner with an immediate idea of how successful things were. The most powerful GPT SDD controller matches the oracle exactly. The most powerful Opus SDD controller achieves 9 out of 15 and consequently beats both the oracle and the most powerful Opus PDD controller in terms of number of solved scenarios, even though it cannot match the oracle's perfect behavioural specifications. The most powerful Opus PDD controller, while still being a valuable analytical tool, solves a completely different set of problems.

The most intriguing thing about this repository, perhaps the whole thesis, is that the gpt_sdd_a version 3 controller perfectly matches the oracle on its event-and-step criteria in all 15 problems. This needs to be stated explicitly as it indicates that, in this lineage at least, a specification-driven workflow can reproduce the benchmark oracle's reference

Namely, for each of the 15 scenarios, the gpt_sdd_a version 3 always experiences the same terminal state (either GOAL_REACHED or TIMEOUT) and has the same number of steps until termination compared to the oracle. In other words, both controllers solve the same set of 7 scenarios and run out of time solving the remaining 8. The latter fact demonstrates an even stricter form of equivalence between the two systems since they yield exactly the same result bits for outcome variables in results.csv.

A second remarkable contrast is that the opus_sdd_a version 1 solves 9 scenarios out of 15 but doesn't behave similarly to the oracle. Specifically, it solves scenarios 6, 8, 10, 14, and 15, which were not solved by the oracle, and fails to solve 3, 11, and 13, which were successfully tackled by both oracle and gpt_sdd_a version 3. It is important because it reveals that generated controllers do not follow a monotonic path to the oracle's behavior.

Another equally interesting PDD-specific alternative is that the opus_pdd_group_b3 version 5 attains a score of 8/15 by completing scenarios 8 and 9, which are not completed by the oracle, but fails scenario 13, which is successfully completed by the oracle. This further indicates that competence in benchmarks is multi-faceted; different types of controllers generated outperform one another in different subsets of the benchmark suite.

It is important to pay attention to which scenarios each algorithm excels at. While scenarios 8 and 9 refer to medium detour problems, scenario 13 is considered a challenging long-detour-budget problem. Hence, this particular Opus PDD controller demonstrates competence when it comes to deciding whether a detour should be undertaken in order to bypass central obstacles and valid gaps, but fails in more complex decision-making processes when it comes to budget constraints. On the other hand, both the oracle and gpt_sdd_a version 3 retain their ability to handle more difficult problems, like the long detour budget, without succeeding in scenarios 8 and 9.

7.4 Final-version behaviour

Final peak performance and final version performance are two separate things.

In their final versions, both gpt_pdd_group_b1 and gpt_pdd_group_b3 are only able to solve 4/15, while the final GPT SDDs solve between 4 and 5 out of 15. In contrast, all three Opus PDD core lineages reach zero performance, whereas the final versions of the Opus SDD core lineages solve 5 to 6 out of 15 problems. In other words, this particular version of Opus experiences a sudden late-game failure under PDD, while SDD continues to function despite degeneration.

This serves as very convincing proof that version drift requires proper interpretation, as it is not merely random noise. A lineage could achieve amazing success in the middle of the process and still perform worse than expected at the end.

The number of successful scenario runs out of a total of 90 for the GPT core family final versions aggregated is 22. The number of successful scenario runs for the Opus core family final versions aggregated is 17 out of 90. GPT thus maintains an advantage in the end-state condition, but it is a small one, and much smaller than might be inferred by the Opus PDD collapse conditions alone since the Opus SDD families remain functionally capable at that point.

7.5 Behavioural differences between the AI sources

The repository offers unique behavioral traits for the two models in question depending on the applied prompting discipline.

First, GPT-5.4 reacts positively to strict specification: its SDD controllers show readiness to completely overwrite behavior. As a result, GPT-5.4 shows high-amplitude adaptation with improvements and setbacks. These improvements include peaks that improve average runnable behavior of the model in

question and an oracle-like convergence on one lineage; setbacks include the occurrence of significant drift.

Second, Claude Opus 4.7 exhibits different properties when prompted in the same artifact. While under PDD prompts Opus may generate useful local heuristics (the best controller achieves a score of 8/15), these improvements prove fleeting, as later Opus PDD versions fail. Meanwhile, while generating stronger controllers under SDD prompts (including the strongest non-target controller with a score of 9/15 in the whole repository), Opus generates significant drift in addition to improvement.

This is an important conclusion from the viewpoint of the dissertation problem statement. Within the two model sources examined here, this suggests that the effect of SDD is not invariant to the underlying model, and claims about SDD's value must therefore specify which model-source they apply to. Specification discipline affects the space of exploration, but the impact on the space varies across the AI system types. While GPT-5.4 employs the richer contract to consider behavioral rewrites of a broader scope that bring numerous improvements but also many regressions, Opus 4.7 is more conservative in its evolution and does not evolve monotonically.

For a fair comparison, one should avoid the temptation to make simplistic claims about the supremacy of certain vendors. The correct way of interpreting the findings would be to recognize the interaction between prompt discipline and model source.

7.6 Drift comparison

The drift result data is where the real engineering story lies.

In terms of the drift results, for the GPT PDD core lineages, the drift results range between 0.276 and 0.467 and there are quite a lot of same comparisons. In contrast, the GPT SDD lineages have very high drift results between 0.933 and 0.956, but with hardly any same comparisons. As such, GPT SDD produces better behavioral change but suffers far greater instability.

As for the Opus PDD core lineages, the drift results are also very high, ranging between 0.686 and 0.790, with a final collapse in success to zero for the final versions. Thus, Opus PDD does not produce anything that can be called stable or weak. On the other hand, the drift results for the Opus SDD lineages are relatively high compared to the negligible drift results, from 0.489 to 0.889.

Two types of counting are present in this dissertation. pairwise_changes.csv has 1215 pairs of consecutive versions per scenario in total across the whole repository, whereas the core subset of interest for the analysis presented below is 900 rows (60 non-initial versions of the core times 15 scenarios), which corresponds to the total number in Appendix C.4. Its counterpart in pairwise_summary.csv has 81 pairs of version summaries for the whole repository. Whenever the percentages below for drift and regression rates pertain to the main body of text, they are based on the core slice of 900 rows of pairwise_changes.csv.

As for aggregate results, the three GPT PDD core lines make up 315 adjacent version scenarios which include 204 same, 46 pure drift, 12 regression and 53 improvement scenarios. The three GPT SDD lines form 135 scenarios where there are 7 same but 52 pure drift, 21 regression and 55 improvement scenarios. As to the Opus PDD core lines, they consist of 315 scenarios with 87 same, 162 pure drift, 29 regression

and 37 improvement scenarios. As for the Opus SDD core lines, they form 135 scenarios which include 45 same, 55 pure drift, 23 regression and 12 improvement scenarios.

The above counts highlight the existence of four types of revision behavior. The GPT-PDD exhibits moderate revision with high preservation. On the other hand, the GPT-SDD has high revision with numerous improvements but also some regressions. The Opus-PDD demonstrates very high pure drift with low long-term preservation. The Opus-SDD lies somewhere in the middle, with higher preservation than the GPT-SDD or the Opus-PDD but with numerous non-improving revisions and some regressions.

This produces one of the most important analytical conclusions in the report:

Drift needs to be decomposed, not just counted.

A single drift total hides whether change was preserved, exploratory, regressive, or genuinely improving. Drift only becomes meaningful when read alongside task success, change-type composition, and integration validity.

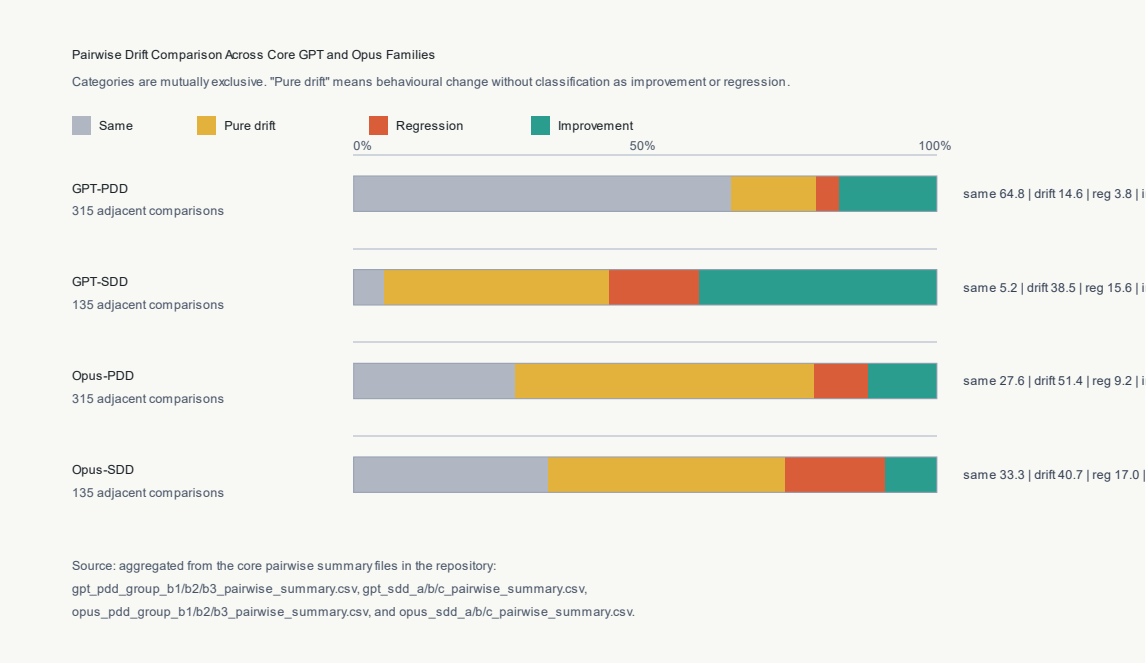

Figure 4. Exclusive pairwise change categories across the core GPT/Opus PDD and SDD lineages. "Pure drift" denotes behaviour change that is neither classified as regression nor improvement.

7.7 What drift means for the dissertation question

The drift data answers a key part of the research question directly. SDD does not reduce drift; in the GPT-5.4 lineages it substantially increases it, raising the drift rate from 0.35 to 0.95. The correct interpretation is that SDD changes the character of drift rather than suppressing it. Under PDD, drift is dominated by preservation (65% same comparisons for GPT PDD) punctuated by incremental improvement. Under SDD, each revision tends to be a larger behavioural rewrite, which means both improvement and regression occur more frequently. The Opus 4.7 pair tells a different story: PDD produces high drift (0.72) with a final collapse to 0/15 in all three core lineages, while SDD preserves more (45 same out of 135) and maintains usable end-state performance. This interaction is the dissertation's main empirical

contribution: the effect of specification discipline on drift is model-contingent, and neither workflow dominates on all criteria simultaneously. Developers who treat drift as a single number will miss this structure entirely; decomposition into same, pure drift, regression, and improvement is necessary to reason about whether iteration is making a codebase better or worse.

7.8 Difficulty and challenge-type patterns

The reason these difficult benchmarks are relevant is that they demonstrate differing tendencies in controller behaviour which may be obscured in simpler tests. In previous summaries from the repository, which involved just GPT models, PDD appears to be comparatively stronger in linear pathfinding and corridor pathfinding tasks compared to SDD, while the latter is comparatively stronger on singular detours and dead-ends. This matches the coding patterns found in the better controllers; PDD tends to develop simple movement heuristics, while SDD tends to develop reasoning about being stuck.

However, the cross-model results add clarity. The better performing Opus PDD controller is able to pass two of the three medium difficulty detour benchmarks which the oracle is unable to solve, showing that opportunistic heuristics may have advantages over a more risk-averse reference controller on certain map designs. However, this success is unable to generate a consistent lineage of controllers, illustrating why these hard and counter-intuitive benchmarks are important.

The artefacts used in the historical prompt support the same assertion. In the early versions of the PDD prompt file, failure instances were still being referred to as collisions. However, in the final data set, failure instances have been presented mainly in terms of timeout following a non-fatal blocked movement change. This may not necessarily be seen as a flaw in the report if it is well-explained.

7.9 Integration failures and repeated old-spec failure

The invalid controller examples represent some of the most convincing results in the collection.

There are controllers that fail due to the absence of the mandatory Controller class. There are other controllers that fail because of their assumption about observation.front_blocked. This latter group is especially significant, since these controllers appear to be reasonable programs and are prone to be overrated during an informal test. However, they are invalid artifacts of the system.

The historical development of the program confirms that this is not an isolated incident. All three original versions of controllers sdd_a, sdd_b, and sdd_c demonstrated the same legacy specification error. Such consistent errors serve as evidence that there can be systematic lineage specification issues even before any behavioral assessment is conducted.

```
approach = gpt_pdd_group_b1
version = 1
scenario_id = 1
event = INVALID_CONTROLLER
error = runtime error: 'Observation' object has no attribute 'front_blocked'
```

Figure 5. Integration failure recorded by the harness for the first GPT-5.4 PDD version that assumed an observation.front_blocked field. The controller is rejected as INVALID_CONTROLLER before any scenario is executed. The same failure mode appeared in the first versions of sdd_a, sdd_b, and sdd_c.

7.10 What the prompt artefacts themselves reveal

Prompt files are not just ancillary documents; rather, they constitute the evidence itself. PDD prompts reflect a process in which performance summaries get re-inputted into the model with relatively soft guidance as to how the output is to be maintained. This is helpful for experimental improvements, but it implies leaving significant aspects of the assignment unarticulated. By contrast, SDD prompts try to write out a binding contract, thereby reducing the amount of inferential work that the model needs to do.

This provides some insight into the empirical phenomenon. When PDD succeeds, it is frequently because the model finds a functional heuristic to work with and retains a substantial portion of it as it makes subsequent modifications. When it fails, either the model makes mistakes in its assumptions about the correct interface or it drifts into another heuristics basin, lacking clear contractual references. In the case of successful GPT-SDD prompts, the success comes from sufficient clarity in the structure provided to the model in order to generate deliberative controller code. The failures in GPT-SDD tend to involve over-corrective behavioural adjustments, rather than interface ambiguity.

It is in this regard that the Opus artefacts become particularly insightful. The best Opus PDD controller implies that even informal prompting may reveal useful behavior even against the deterministic standard. However, the subsequent Opus PDD collapse suggests that, in this corpus, informal feedback on its own did not maintain such gains across later revisions. The Opus SDD artefacts illustrate an alternative trade-off between explicitness and effectiveness: explicit specifications may generate highly efficient controllers initially; yet their further modifications do not guarantee monotonic improvement and may actually involve severe deterioration. It is for this reason that the dissertation needs to avoid simplistic claims such as "specifications are always good" and "prompting is sufficient."

8. Discussion

8.1 Main answer to the research question

SDD improves structural discipline and raises the ceiling of achievable controller quality for both model sources. It does not reduce behavioural drift. For GPT-5.4, SDD dramatically increases drift (rate 0.95 vs 0.35 for PDD) because more explicit contracts induce larger rewrites; this enables oracle-equivalent performance on one lineage while creating substantial regression risk across others. For Opus 4.7, SDD improves peak and final-version performance over PDD but still produces a drift rate of 0.67. The data supports no general claim that more specification equals less drift.

Two findings deserve explicit restatement as the core contribution of this dissertation. First, SDD improves structure but not drift: the two properties are independent, and evaluating AI-assisted workflows on performance alone misses the stability problem. Second, drift must be decomposed: a single aggregate rate obscures whether revision is making a codebase better or worse, and only the decomposed view (same / pure drift / regression / improvement) reveals the difference between GPT-SDD’s high-amplitude adaptation and Opus-PDD’s slow drift into total collapse. The relationship between prompting discipline, model behaviour, and engineering quality is contingent on both the workflow and the underlying model.

8.2 What we can conclude from the different drift patterns

The drifts are no mere byproduct. Rather, they form the key engineering insights of the research work.

In the case of GPT-5.4, large drift under SDD appears to be the cost incurred for better adaptation performance. In effect, while the model gets smarter, it also becomes more prone to adjusting controller structure across versions. This makes for both oracle-like operation and a significant potential for regression.

As for Opus 4.7, PDD is observed to produce some good intermediate controllers and also some very bad controllers in the latter part of the experiment. With SDD, meanwhile, the best non-target controller is found and better retention is achieved, but the pairwise summary results are also riddled with non-improvements and a noticeable regression problem.

The same comparison shows why "behavioural reliability" cannot be summarized by a single measure. Counting success alone would give an advantage to Opus SDD on sheer number of scenarios solved and GPT SDD on oracle-equivalence. Counting drift alone would cast both GPT-SDD and Opus-PDD in terms of chaos, while obscuring the structure of the change itself. The true engineering story is told only by looking at success, interface validity, and drift together.

This insight generalizes into an argument about software engineering principles for AI-assisted development: specification discipline and change discipline are both required for AI-assisted development. Specification discipline helps align interfaces, but if regeneration repeatedly rewrites the entire artifact, then behavioral stability remains precarious.

8.3 Why determinism was essential

In the absence of determinism, the above arguments would lose much of their force. The dissertation can assert drift from version to version without fear, knowing that each scenario has its own canonical trace. If there is a difference between gpt_sdd_a version 3 and version 4, it lies in differences in controller logic, not randomness in simulation runs.

This is a very powerful strength of the methodology employed by this entire research endeavor. It transforms AI-assisted development from an informal prompter into a rigorous engineering comparison.

It also distinguishes the dissertation from a standard model benchmarking experiment. A standard model benchmarking experiment would focus on determining which of two systems scores higher in a set of predetermined tests. The current research takes the much more software-engineering-centric approach of asking how a piece of code degrades in quality, validity, and preservation of behavior when iterated upon multiple times.

8.4 How the results relate to maintainability

In general, the better the SDD controllers are, the more organized they are. They make sure that they do not hide the historical state and past blockage attempts; they have a finite memory and show their internal states in a readable way. This makes inspection easier. However, later generations of SDD show that there is a price for an increased controller complexity: if the internal algorithm becomes complex enough, regeneration will affect good behavior negatively.

Opus demonstrates a related point. It is possible for the lineage of controllers to remain syntactically valid yet become useless due to behavioral problems. Thus, maintainability involves much more than just keeping the form intact.

8.5 Why the project is difficult and worthwhile

The importance of this project is that it needed more than just prompting for models. The project entailed the design of a deterministic simulator, setting up benchmark test cases, creation of a harness for loading and evaluating the program, determination of the failure representations, capturing traces, doing drift analysis, and then analysis of various controllers derived from different models. In essence, this is an empirical software engineering artifact as opposed to an exercise.

8.6 Practitioner implications

For engineers who are using AI assistance in iterative code development today, the findings in this dissertation suggest four practical habits. First, treat the first-generation output as an integration test before it is a functional test: structural failures (missing interfaces, invented fields) are the most common failure mode, and they are catchable before any behavioural benchmark runs. Second, maintain an explicit contract for what the model is allowed to assume about its environment; the observation contract used in this benchmark (a fixed set of permitted input fields) eliminated an entire class of regression that otherwise repeats across SDD-a, SDD-b, and SDD-c lineages. Third, do not treat specification discipline as a substitute for behavioural measurement: SDD prompts reduced structural failures but did not monotonically reduce behavioural drift, and regression-heavy revisions remained common. Fourth, evaluate AI-assisted code at the trace level rather than the pass/fail level where possible; pass/fail aggregates hide the direction of change, and direction of change is the signal that predicts whether iteration is making the codebase better or worse.

Two additional operational habits follow directly from the drift decomposition. Fifth, install regression guards as an explicit step in the revision loop: before accepting a new controller version, re-run the benchmark subset that the previous version already passed and block acceptance if any scenario regresses from GOAL_REACHED to TIMEOUT or to INVALID_CONTROLLER. The pairwise drift classification in §5.8 operationalises this check and flags the two kinds of change that matter most, regression and pure drift on previously-solved scenarios. Sixth, prefer controlled local edits to full rewrites: the Opus SDD and GPT SDD lineages show that large revisions improve some scenarios while silently regressing others, so when a specific scenario needs fixing, scope the change to the observation-to-action logic responsible for that scenario rather than regenerating the controller from the full specification. Both habits treat AI-generated controller code as a codebase under maintenance rather than as a disposable artefact, which is the mindset this dissertation argues is necessary for iterative AI-assisted development to scale.

9. Limitations And Threats To Validity

9.1 Internal validity

The most significant internal validity threat is chronological ordering: PDD lineages were generated before SDD lineages, so the SDD work benefited from greater researcher familiarity with the benchmark

and a more mature test harness. This ordering cannot be undone, and it is a genuine confound on the performance advantage observed for SDD.

There are also some questions regarding which branches belong to the main study; the repeated a/b/c families are a cleaner comparison unit than the full repository.

A further threat, which applies specifically to the model-source comparison, is the lack of API-level control over generation parameters. All controllers were generated through the consumer chat interfaces of GPT-5.4 and Claude Opus 4.7; temperature, top-p, and system-prompt settings were not under experimental control and may have differed across sessions or changed with platform updates. This means observed differences between GPT-5.4 and Opus 4.7 could partly reflect default sampling differences rather than fundamental model behaviour. The archived controller source files allow the benchmark results to be reproduced exactly, but re-generating the lineages from the prompts is not bit-reproducible for this reason.

The per-cell lineage count of n = 3 also limits statistical claims. With three lineages per model-workflow cell, it is not possible to distinguish between systematic effects and within-cell noise using standard hypothesis tests. The late-stage Opus PDD collapse and the gpt_sdd_a oracle match are each observed in a single lineage; they may reflect systematic tendencies of the respective workflows, or they may be outliers that would not replicate at higher n. The appropriate response is to treat the quantitative findings as hypothesis-generating rather than confirmatory, and to prioritise replication at $n \geq 10$ per cell in future work (§10.2).

Another aspect of internal validity relates to the fact that the prompts differed by design, but not in the trivial manner of wording only. Insofar as SDD specifies the system contract in greater detail, it imposes stronger behavioural constraints on the model. Some difference thus lies not only in better wording, but in the underlying information structure provided to the model.

9.2 Construct validity

The operationalisation of behavioural drift uses outcome events, step counts, and trace equivalence. This is well-suited to the deterministic benchmark, but it is one possible operationalisation among several. An analysis based on edit-distance between action sequences, for instance, would capture structural trajectory changes that the current outcome-based classification treats as same.

As noted in §4.3, blocked movement is recorded as TIMEOUT rather than COLLISION in the final dataset following the non-fatal collision change. This is documented in the data schema (§4.8) and is relied upon consistently throughout the analysis, but it means the COLLISION value in the terminal-event enum is unused in the core results, which could mislead a reader who inspects the schema without reading the methodology.

9.3 External validity

The benchmark remains an abstract grid world scenario. It makes no allowances for noisy sensor readings, continuous dynamics, physical actuation, or uncertainty about the hardware components used. Conclusions of the dissertation should thus be made relative to the process of software engineering assisted by AI techniques within the controlled environment.

9.4 Reproducibility gaps

The repository is strong on benchmark reproducibility: prompt artefacts, all 107 controller source files, results.csv, and the analysis scripts are committed and commit-pinned. The remaining gap is generation-process reproducibility. Because controllers were produced through consumer chat interfaces rather than the API, the exact sampling parameters (temperature, top-p, any system-level instructions applied by the platform) are unknown and were not logged. Rejected regenerations—prompts that produced invalid or obviously broken controllers before the version that was committed—are also not documented. This means that the sequence of prompts in the repository represents the accepted lineage, not the full interaction history. Future work should log all generation attempts, including rejections, and should use API access with fixed sampling parameters to close this gap.

10. Future Work

The next step is not merely to experiment with another model. Instead, the next step should be to exert greater control over the change process. The next generation of experiments should contrast full-file regeneration versus patch-based regeneration, incorporate clear regression guards, and explore whether a more nuanced set of edit constraints can retain the benefits of SDD while mitigating its most egregious instabilities.

Two other comparisons are likewise natural extensions. In the first place, the initial experiment was designed with the expectation of contrasting the PDD static prompt, PDD adaptive prompt, and SDD methods as three distinct treatments. Completion of this comparison would clarify the independent influence of feedback and specification discipline. Secondly, a later experiment might contrast lightweight manual SDD as conducted here and more elaborate tool-supported approaches like Spec Kit.

An additional avenue to explore is model-specific specification design. In one way of thinking about the difference between the GPT and Opus models, the issue might not be one where one model is better than the other, but rather how different models can react differently to the same type of explicit contract. Thus, future research could use the same SDD template and compare it against a model-specific specification template, all the while controlling for the benchmark and testing harness.

Lastly, the entire testing environment could be moved out of the robotic grid world into an entirely different software environment altogether.

10.1 Richer multi-agent prompting workflows

This particular PDD/SDD classification is intentionally simplistic in nature, with both workflows involving a single-agent, single-turn prompting methodology. One obvious way to extend the study would be to consider multi-agent specification-driven development where the specification generation and its implementation using the specification are handled by two separate models. This is straightforwardly done on the same benchmark architecture without any changes other than the prompting workflow discussed in §§5.4 and 5.5. It would also be interesting to see if there is less chance of regression in this case compared to SDD (criterion 5, §2.7).

10.2 Larger lineage counts for statistical power

Given three lineages per each model/workflow combination, §9.1 observes that it is not possible to differentiate between noise and certain features such as the late-stage failure of the Opus PDD or the oracle match for GPT SDD using traditional hypothesis testing. Using ten or even more lineages per each model/workflow combination would allow for the use of non-parametric tests based on drift rate (e.g. Mann-Whitney U test based on regression count per lineage). The marginal additional cost is just computational power.

10.3 Beyond the grid-world: SWE-bench-style real repositories

The grid world intentionally removes the issue of measuring drift from the ambiguities present in the specification of a real-world problem domain. The other direction, therefore, would be to implement the drift decomposition methodology into a repository benchmark such as SWE-bench (Jimenez et al., 2024) or LiveCodeBench (Jain et al., 2024). In terms of analysis, this can be accomplished by applying the results discussed in §5.8 since, for adjacent-version classification, it is necessary to have a stable output function, along with the ability to retrain the model on its output.

10.4 Formal specifications as the SDD target

SDDs that were used in this thesis are natural language contracts expressed in the structured form (refer to Appendix A). The better approach would have been to write down formal contract specifications, e.g. in TLA+ or Alloy language specifying legal observation access patterns and action sequences, and require from the modeler to produce a controller meeting such a specification. This would provide closure with respect to the discussed formal verification literature in §2.5 (Luckcuck et al., 2020), and would also enable automated specification compliance check before running a scenario, thus making the validation criterion discussed in §2.7 stronger.

11. Conclusion

11.1 Summary of achievements

This section describes how each of the aims described in §3.2 maps to an achievement. The mappings allow for an audit of the delivery claims made: each aim is mapped either directly to an outcome or has been descoped with reasoning provided.

Aim (from §3.2)	Status	Evidence
A1. Build a deterministic benchmark capable of measuring AI-assisted code behaviour over iteration.	**Delivered**	§4.3 simulator; R1–R4 in §3.4 delivered; 1605 reproducible runs (§6.1).
A2. Define deterministic separation between PDD and SDD workflows so the variable under test is the prompting discipline.	**Delivered**	§5.3 separation rules; §5.4 and §5.5 workflow descriptions; fairness corrections documented in §5.7.

A3. Evaluate PDD and SDD across repeated lineages on two model sources.	**Delivered**	§6.2 core families; four model-by-workflow cells each with three lineages; Appendix C.1 per-version matrix.
A4. Analyse drift as decomposition, not as a single aggregate, to avoid hiding regression under improvement.	**Delivered**	§5.8 metric definitions; §7.6 drift comparison; Appendix C.4 table; §1.1 framed as a contribution.
A5. Interpret the findings as software engineering evidence rather than as benchmark scores.	**Delivered**	§8 discussion explicitly argues maintainability and integration reliability implications; §2.1 framing.
A6. Produce reproducible artefacts.	**Delivered**	15 passing tests via pytest tests -q (§6, C.6); commit-pinned reproduction (C.6); schema documented in §4.8.
A7. Provide integration-failure evidence capturing contract violations distinctly from behavioural failures.	**Delivered**	§7.9 invalid-controller analysis with Figure 5 screenshot; repeated old-spec failure across sdd_a, sdd_b, sdd_c documented.
A8. Real-time collaborative multi-agent generation.	Descoped	R14 descoped; the empirical story was already rich enough at the current scale. See §10.1.
A9. Statistical significance testing across lineages.	Partial	Lineage counts (n=3 per cell) are too small for robust tests; limitation acknowledged in §9.1 and reframed as future work in §10.3.

Of the nine goals, seven of them have been fully met, one of them has been partially addressed but the rest is reserved for future work, while another goal has been explicitly decoupled from our work. All three of the empirical contributions described in §1.1, namely the deterministic benchmarking framework, no correlation between specification discipline and drift, and drift decomposition framework, have direct equivalents to deliverables A1–A4.

This dissertation examined Prompt-Driven Development and Specification-Driven Development for iterative AI-assisted controller generation in a deterministic grid-world benchmark. Its contribution is not the highest-scoring controller but a framework for treating AI-generated revisions as versioned engineering artefacts that can be audited for interface validity, behavioural performance, and drift across iterations.

The qualified answer to the research question is this: SDD improves structural discipline and peak performance but does not reduce behavioural drift. For GPT-5.4, SDD achieves oracle-equivalent behaviour on one lineage (gpt_sdd_a v3) at the cost of a drift rate of 0.95. For Opus 4.7, SDD produces the strongest non-target controller in the repository (opus_sdd_a v1, 9/15) and better end-state retention than PDD, but drift and regression persist across all three SDD lineages. In both cases, SDD transforms the character of unreliability from interface ambiguity into contract-compliant rewrites—a meaningful improvement, but not a solution to the drift problem.

The broader lesson is that productive discussion of AI-assisted software development cannot focus only on whether the first generated version works. It must ask what happens over iterations: whether the

interface is preserved, whether earlier gains are retained, and whether the revision process itself is observable and accountable. The two contributions that answer those questions are the drift-decomposition framework—which distinguishes regression, improvement, and pure behavioural change—and the finding that SDD shifts but does not eliminate the drift problem. Advancing AI-assisted development from informal prompting to disciplined engineering requires explicit contracts, deterministic evaluation, trace-level measurement, and active regression guards alongside specification discipline.

References

Anthropic (2026) *Introducing Claude Opus 4.7*. 16 April 2026. Available at: https://www.anthropic.com/research/claude-opus-4-7

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q. and Sutton, C. (2021) 'Program Synthesis with Large Language Models', arXiv:2108.07732. Available at: https://arxiv.org/abs/2108.07732

Chen, M. et al. (2021) 'Evaluating Large Language Models Trained on Code', *arXiv*. Available at: https://arxiv.org/abs/2107.03374

Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S. and Zhang, J.M. (2023) 'Large Language Models for Software Engineering: Survey and Open Problems', in Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE 2023), pp. 31–53. Available at: https://doi.org/10.1109/ICSE-FoSE59343.2023.00008

GitHub (2026) *Spec Kit: Toolkit to Help You Get Started with Spec-Driven Development*. Available at: https://github.com/github/spec-kit

ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering - Life Cycle Processes - Requirements Engineering*. Available at: https://www.iso.org/standard/72089.html

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K. and Stoica, I. (2024) 'LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code', arXiv:2403.07974. Available at: https://arxiv.org/abs/2403.07974

Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. and Narasimhan, K. (2024) 'SWE-bench: Can Language Models Resolve Real-World GitHub Issues?', in Proceedings of the 12th International Conference on Learning Representations (ICLR 2024). Available at: https://openreview.net/forum?id=VTF8yNQM66

Lehman, M.M. (1980) 'Programs, Life Cycles, and Laws of Software Evolution', Proceedings of the IEEE, 68(9), pp. 1060–1076. Available at: https://doi.org/10.1109/PROC.1980.11805

Liu, J., Xia, C.S., Wang, Y. and Zhang, L. (2023) 'Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation', Advances in Neural Information Processing Systems, 36. Available at: https://arxiv.org/abs/2305.01210

Luckcuck, M., Farrell, M., Dennis, L.A., Dixon, C. and Fisher, M. (2020) 'Formal specification and verification of autonomous robotic systems: A survey', *ACM Computing Surveys*, 52(5). Available at: https://doi.org/10.1145/3342355

OpenAI (2026) *Introducing GPT-5.4*. 5 March 2026. Available at: https://openai.com/index/introducing-gpt-5-4/

Sallou, J., Durieux, T. and Panichella, A. (2024) 'Breaking the Silence: the Threats of Using LLMs in Software Engineering', in Proceedings of the 2024 ACM/IEEE 46th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER 2024), pp. 102–106. Available at: https://doi.org/10.1145/3639476.3639764

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F. and Dennison, D. (2015) 'Hidden Technical Debt in Machine Learning Systems', in Advances in Neural Information Processing Systems 28 (NIPS 2015), pp. 2503–2511. Available at: https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

Vaithilingam, P., Zhang, T. and Glassman, E.L. (2022) 'Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models', *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. Available at: https://doi.org/10.1145/3491101.3519665

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. and Zhou, D. (2022) 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models', Advances in Neural Information Processing Systems, 35, pp. 24824–24837. Available at: https://arxiv.org/abs/2201.11903

Wohlin, C. et al. (2024) *Experimentation in Software Engineering*. 2nd edn. Springer. Available at: https://doi.org/10.1007/978-3-662-69306-3

Zhuo, T.Y. et al. (2024) 'ProSA: Assessing and Understanding the Prompt Sensitivity of LLMs', *Findings of the Association for Computational Linguistics: EMNLP 2024*. Available at: https://aclanthology.org/2024.findings-emnlp.108/

Appendix A — Extended Prompt Artefact Excerpts

This appendix includes sample prompt artifacts that were utilized in the GPT and Opus PDD and SDD controller runs.

The following GPT prompt artifacts were directly extracted from the prompt files stored in the repository. Since the prompts for the Opus controller runs were generated via the browser-based interface, the prompt artifacts here were extracted from the interaction log file for those runs.

A.1 GPT PDD representative prompt artefact

Repository source: Project/controllers/pdd_group_b3/prompt_v3.txt

```
Write Python code for a grid robot controller. The robot is trying to reach a goal
position. It knows its current position, heading, and the goal position. It can
choose to move forward, turn left, turn right, or wait.

Use these imports: from gridbot.sim.actions import Action, Heading
Define a class Controller with a method act(self, observation) -> Action.
```

```
The observation has: observation.position, observation.heading, observation.goal.
Use the existing enum names exactly: Heading.N, Heading.E, Heading.S, Heading.W
and Action.FORWARD, Action.TURN_LEFT, Action.TURN_RIGHT, Action.WAIT.

Previous controller results for this chain:
- Successes: 4/15
- Collisions: 11/15
- Timeouts: 0/15
- Invalid runs: 0/15
- Average steps: 4.733

Successful scenarios:
- 1/easy_open_short_path: event=GOAL_REACHED, steps=4
- 3/easy_l_wall_one_turn: event=GOAL_REACHED, steps=10
- 6/medium_straight_narrow_corridor: event=GOAL_REACHED, steps=7
- 11/hard_maze_one_dead_end: event=GOAL_REACHED, steps=13

Failing scenarios:
- 2/easy_single_obstacle_detour: event=COLLISION, steps=2
- 4/easy_corner_goal_blocked: event=COLLISION, steps=4
- 5/easy_two_isolated_obstacles: event=COLLISION, steps=2
- 7/medium_s_shaped_path: event=COLLISION, steps=5
- 8/medium_central_obstacle: event=COLLISION, steps=2
- 9/medium_single_valid_detour: event=COLLISION, steps=4
- 10/medium_dead_end_corridors: event=COLLISION, steps=2
- 12/hard_u_trap_goal: event=COLLISION, steps=4
- 13/hard_long_detour_budget: event=COLLISION, steps=8
- 14/hard_two_dead_ends: event=COLLISION, steps=3
- 15/hard_move_away_corridor: event=COLLISION, steps=1

Improve the controller so it is more successful and more efficient. Prioritise
reaching the goal on more scenarios. If a scenario already succeeds, try not to
make it worse. Output only Python code.
```

A.2 GPT SDD representative prompt artefact

Repository source: Project/controllers/sdd_a/prompt_v4.txt

```
The previous controller is runnable and currently the strongest SDD chain, but
it still shows unstable drift and still fails on:
- 6 medium_straight_narrow_corridor
- 7 medium_s_shaped_path
- 8 medium_central_obstacle
- 9 medium_single_valid_detour
- 10 medium_dead_end_corridors
- 12 hard_u_trap_goal
- 14 hard_two_dead_ends
- 15 hard_move_away_corridor

It currently succeeds on:
- 1 easy_open_short_path
- 2 easy_single_obstacle_detour
```

- 3 easy_l_wall_one_turn
- 4 easy_corner_goal_blocked
- 5 easy_two_isolated_obstacles
- 11 hard_maze_one_dead_end
- 13 hard_long_detour_budget

Regenerate the controller and follow this specification exactly.

1. File Contract
The output must be a single Python file.
The file must contain only controller code.
The file must import exactly:

```
from gridbot.sim.actions import Action, Heading
```

The file must define exactly:

```
class Controller:
    def act(self, observation) -> Action:
        ...
```

2. Observation Contract
Use only:
- observation.position
- observation.heading
- observation.goal

Do not access any other observation fields.

3. Action Contract
Return only:
- Action.FORWARD
- Action.TURN_LEFT
- Action.TURN_RIGHT
- Action.WAIT

4. Heading Contract
Use only:
- Heading.N
- Heading.E
- Heading.S
- Heading.W

5. Coordinate Convention
- Heading.E increases x
- Heading.W decreases x
- Heading.S increases y
- Heading.N decreases y

6. Progress-Inference Rule
Infer blocked forward movement only when:

```
- the previous action was Action.FORWARD, and
- the current position is unchanged from the previous position

7. Behaviour Requirements
If already at the goal, return Action.WAIT.
Otherwise, attempt to reach the goal deterministically.

Preserve the existing strengths of this chain on easy scenarios and on:
- 11 hard_maze_one_dead_end
- 13 hard_long_detour_budget

Improve specifically:
- re-entering corridors cleanly after detours
- handling narrow corridors without overcommitting to remembered blocked edges
- committing to longer detours around central obstacles and dead ends
- moving away from the goal temporarily when needed

8. Planning Constraints
You may use bounded deterministic memory.
You may use bounded local planning or bounded search.
If you use inferred blocked moves, keep them bounded and episode-local.
Do not let remembered blocked edges cause over-pruning of valid corridor moves.
Do not use unbounded global search.

9. Recovery Constraints
If using recovery mode:
- use a clear entry condition
- use a clear exit condition
- commit for multiple steps before abandoning the recovery
- allow switching side only after repeated failed progress
- avoid short oscillations near obstacle corners

10. Output Requirements
Output only Python code.
Do not include explanations.
```

A.3 Opus PDD representative prompt artefact

Source: recorded interactive generation log, Opus PDD clarified follow-up prompt

```
The previous controller did not improve overall. It still solved only a small
number of scenarios, and one likely issue is that the environment contract was
being inferred incorrectly.

Please write one more improved deterministic controller version using the same
observation format.

Important environment details:
- The observation has: observation.position, observation.heading, observation.goal
- observation.position and observation.goal are (x, y) tuples
- Use the project's coordinate convention: Heading.E increases x, Heading.W
```

```
  decreases x, Heading.S increases y, and Heading.N decreases y
- If the controller returns Action.FORWARD and the robot stays in the same
  position on the next step, that means it hit a wall, obstacle, or boundary
  and did not move

The controller should use this correctly and try to improve performance on
simple detours, corridors, dead ends, and longer obstacle routes.

Use these imports: from gridbot.sim.actions import Action, Heading
Define a class Controller with a method act(self, observation) -> Action.
Use the existing enum names exactly: Heading.N, Heading.E, Heading.S, Heading.W
and Action.FORWARD, Action.TURN_LEFT, Action.TURN_RIGHT, Action.WAIT.
```

A.4 Opus SDD representative prompt artefact

Source: recorded interactive generation log, Opus SDD initial specification prompt

```
Write a deterministic Python controller for the grid robot benchmark and follow
this specification exactly.

1. File Contract
The output must be a single Python file.
The file must contain only controller code.
The file must import exactly:

    from gridbot.sim.actions import Action, Heading

The file must define exactly:

    class Controller:
        def act(self, observation) -> Action:
            ...

2. Observation Contract
Use only:
- observation.position
- observation.heading
- observation.goal

Do not access any other observation fields.

3. Action Contract
Return only:
- Action.FORWARD
- Action.TURN_LEFT
- Action.TURN_RIGHT
- Action.WAIT

4. Heading Contract
Use only:
- Heading.N
```

```
- Heading.E
- Heading.S
- Heading.W

5. Coordinate Convention
- Heading.E increases x
- Heading.W decreases x
- Heading.S increases y
- Heading.N decreases y

6. Environment Rule
If the previous action was Action.FORWARD and the current position is unchanged,
that means the attempted forward move was blocked by a wall, obstacle, or
boundary.

7. Behaviour Requirements
If already at the goal, return Action.WAIT.
Otherwise, attempt to reach the goal deterministically.
The controller may use bounded internal state.
The controller should infer failed forward progress only from position change
across steps.
The controller should try to move toward the goal, but it must be able to
recover from blocked forward movement.

8. Stability Requirements
The controller must be deterministic.
Do not use randomness.
Do not use unbounded global search.
Keep any internal memory bounded and episode-local.

9. Output Requirements
Output only Python code.
Do not include explanations.
```

Appendix B — Scenario Reference Table

The full scenario reference table is provided as part of Appendix C to keep all experimental evidence together. See Appendix C.5 for the complete table of 15 scenarios, including scenario IDs, names, difficulty tiers, challenge types, grid dimensions, start and goal cells, and step budgets.

Appendix C — Experimental Evidence and Reproducibility

Appendix

This appendix gathers all the data used in Chapter 7 such that each numerical result in the text can be traced to an entry in the rows/columns below. Appendix C.1 shows the complete version-by-version × scenario result matrix; C.2 captures results of final versions for all lineages; C.3 analyzes the best-performing final version for each model workflow combination; C.4 breaks down adjacent version drift and contains a sanity check against the body; C.5 presents the set of scenarios; C.6 provides a regression transition example; and C.7 outlines reproducibility instructions.

Data-source notes

The tables presented in this appendix were obtained from three artefacts that were exported from the project repository: per_version_rows, final_version_rows, and scenario_matrices. There are two notable limitations to this data set:

- The scenario_matrices export retains all three types of outcomes for each scenario that have been used in the repository: success, timeout, and invalid_controller. The scenario_matrices is still an abstract view of the results and does not contain the complete trace information. Thus, if one wants to make any statements regarding the routes, drifts, and actions, they should consult results.csv and pairwise_changes.csv.
- The drift decompositions in C.4 have been derived using the version level outcome_note field (same / improved / drifted / regressed). Since this categorization is coarse-grained with respect to lineages and not for each individual scenario, the resulting subtotals are different from the subtotals obtained by per-scenario pairwise analysis reported in §7.6 of the paper. This difference is highlighted in the table in C.4 and not masked like in the latter case.

C.1 Full per-version × per-scenario outcome matrix

Each row is one controller version; each of the 15 right-hand columns is one benchmark scenario. Cell notation: S (n) denotes GOAL_REACHED in n steps; T (n) denotes TIMEOUT after n steps; I denotes an invalid controller (failed to load or used an unsupported observation field); an em-dash denotes no recorded result.

Lineage	Ver	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15
GPT-5.4 PDD																
gpt_pdd_group_a	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_a	v2	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_a	v3	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_a	v4	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_a	v5	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_a1	v1	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_a1	v2	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_a1	v3	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_a2	v4	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_a2	v5	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_a3	v6	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_a3	v7	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b1	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_b1	v2	T (4)	T (24)	T (28)	T (16)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b1	v3	T (4)	T (24)	T (28)	T (16)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)

Lineage	Ver	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15
gpt_pdd_group_b1	v4	T (4)	T (24)	T (28)	T (16)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b1	v5	T (4)	T (24)	T (28)	T (16)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b1	v6	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b1	v7	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b1	v8	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b2	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_b2	v2	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b2	v3	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b2	v4	T (4)	T (24)	T (28)	T (26)	T (30)	T (9)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b2	v5	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b2	v6	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b2	v7	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b2	v8	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b3	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_pdd_group_b3	v2	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b3	v3	T (4)	T (24)	T (10)	T (14)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)

Lineage	Ver	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15
gpt_pdd_group_b3	v4	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b3	v5	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b3	v6	T (4)	T (24)	T (18)	T (13)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b3	v7	T (4)	T (24)	T (28)	T (15)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_pdd_group_b3	v8	T (4)	T (24)	T (10)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
GPT-5.4 SDD																
gpt_sdd	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_sdd_a	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_sdd_a	v2	T (4)	T (24)	T (10)	T (12)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (13)	T (24)	T (26)	T (28)	T (22)
gpt_sdd_a	v3	T (4)	T (10)	T (18)	T (15)	T (12)	T (15)	T (20)	T (20)	T (18)	T (20)	T (17)	T (24)	T (19)	T (28)	T (22)
gpt_sdd_a	v4	T (4)	T (12)	T (10)	T (17)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (19)	T (24)	T (26)	T (28)	T (22)
gpt_sdd_b	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_sdd_b	v2	T (4)	T (17)	T (20)	T (15)	T (19)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (24)	T (22)
gpt_sdd_b	v3	T (4)	T (22)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_sdd_b	v4	T (4)	T (22)	T (28)	T (15)	T (27)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_sdd_c	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
gpt_sdd_c	v2	T (4)	T (14)	T (15)	T (15)	T (16)	T (7)	T (20)	T (20)	T (18)	T (20)	T (19)	T (24)	T (26)	T (28)	T (22)

Lineage	Ver	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15
gpt_sdd_c	v3	T (4)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
gpt_sdd_c	v4	T (4)	T (24)	T (27)	T (15)	T (30)	T (10)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
Opus 4.7 PDD																
opus_pdd	v1	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd	v2	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd	v3	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd	v4	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd	v5	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a	v1	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a	v2	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a	v3	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a	v4	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a	v5	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a1	v1	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a1	v2	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)

Lineage	Ver	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15
opus_pdd_group_a1	v3	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a2	v4	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a2	v5	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a3	v6	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_a3	v7	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v1	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v2	T (4)	T (20)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v3	T (4)	T (20)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v4	T (4)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (16)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v5	T (4)	T (10)	T (10)	T (10)	T (22)	T (15)	T (20)	T (20)	T (18)	T (20)	T (17)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v6	T (4)	T (10)	T (10)	T (10)	T (22)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v7	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b1	v8	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b2	v1	T (4)	T (24)	T (28)	T (26)	T (30)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b2	v2	T (4)	T (17)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)

Lineage	Ver	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15
opus_pdd_group_b2	v3	T (4)	T (17)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b2	v4	T (4)	T (13)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b2	v5	T (4)	T (9)	T (21)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b2	v6	T (4)	T (10)	T (10)	T (10)	T (19)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b2	v7	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b2	v8	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b3	v1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
opus_pdd_group_b3	v2	T (4)	T (13)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b3	v3	T (4)	T (20)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b3	v4	T (4)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b3	v5	T (4)	T (10)	T (10)	T (10)	T (14)	T (15)	T (20)	T (14)	T (15)	T (20)	T (20)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b3	v6	T (4)	T (10)	T (10)	T (10)	T (14)	T (15)	T (20)	T (16)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b3	v7	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_pdd_group_b3	v8	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
Opus 4.7 SDD																
opus_sdd	v1	T (20)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)

Lineage	Ver	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15
opus_sdd_a	v1	T (4)	T (10)	T (28)	T (15)	T (12)	T (7)	T (20)	T (19)	T (18)	T (13)	T (24)	T (24)	T (26)	T (20)	T (21)
opus_sdd_a	v2	T (4)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_sdd_a	v3	T (4)	T (10)	T (28)	T (15)	T (12)	T (7)	T (20)	T (19)	T (18)	T (13)	T (24)	T (24)	T (26)	T (20)	T (22)
opus_sdd_a	v4	T (4)	T (13)	T (18)	T (15)	T (15)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_sdd_b	v1	T (4)	T (10)	T (18)	T (15)	T (12)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (20)	T (22)
opus_sdd_b	v2	T (4)	T (24)	T (28)	T (26)	T (30)	T (15)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_sdd_b	v3	T (4)	T (9)	T (28)	T (26)	T (30)	T (12)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (19)
opus_sdd_b	v4	T (4)	T (9)	T (28)	T (26)	T (30)	T (12)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (19)
opus_sdd_c	v1	T (4)	T (10)	T (18)	T (15)	T (12)	T (7)	T (20)	T (20)	T (18)	T (13)	T (24)	T (24)	T (26)	T (20)	T (22)
opus_sdd_c	v2	T (4)	T (10)	T (18)	T (15)	T (12)	T (7)	T (20)	T (20)	T (18)	T (13)	T (24)	T (24)	T (26)	T (20)	T (22)
opus_sdd_c	v3	T (4)	T (10)	T (18)	T (15)	T (12)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)
opus_sdd_c	v4	T (4)	T (10)	T (19)	T (15)	T (16)	T (7)	T (20)	T (20)	T (18)	T (20)	T (24)	T (24)	T (26)	T (28)	T (22)

Note. *Source: scenario_matrices in the appendix data export. Header row repeats on each page.*

C.2 Final-version performance summary

One row per lineage. The solved scenario IDs are taken from the final recorded version of each lineage; the counts add up to the success column. This table supports the §7.4 claim that the three Opus 4.7 PDD core lineages end at 0/15 and that Opus 4.7 SDD retains meaningful competence through its final versions.

Lineage	Final	Success	Timeouts	Invalid	Mean steps	Solved scenario IDs
GPT-5.4 PDD						
gpt_pdd_group_a	v5	0/15	0	15	0.0	—
gpt_pdd_group_a1	v3	4/15	11	0	19.5	1, 3, 6, 11
gpt_pdd_group_a2	v5	0/15	0	15	0.0	—
gpt_pdd_group_a3	v7	2/15	13	0	21.4	1, 6
gpt_pdd_group_b1	v8	4/15	11	0	19.5	1, 3, 6, 11
gpt_pdd_group_b2	v8	0/15	15	0	23.0	—
gpt_pdd_group_b3	v8	4/15	11	0	19.5	1, 3, 6, 11
GPT-5.4 SDD						
gpt_sdd	v1	0/15	0	15	0.0	—
gpt_sdd_a	v4	5/15	10	0	19.0	1, 2, 3, 4, 11
gpt_sdd_b	v4	5/15	10	0	20.3	1, 2, 4, 5, 6
gpt_sdd_c	v4	4/15	11	0	20.8	1, 3, 4, 6
Opus 4.7 PDD						
opus_pdd	v5	0/15	15	0	23.0	—
opus_pdd_group_a	v5	0/15	15	0	23.0	—
opus_pdd_group_a1	v3	0/15	15	0	23.0	—
opus_pdd_group_a2	v5	0/15	15	0	23.0	—
opus_pdd_group_a3	v7	0/15	15	0	23.0	—
opus_pdd_group_b1	v8	0/15	15	0	23.0	—
opus_pdd_group_b2	v8	0/15	15	0	23.0	—
opus_pdd_group_b3	v8	0/15	15	0	23.0	—
Opus 4.7 SDD						

Lineage	Final	Success	Timeouts	Invalid	Mean steps	Solved scenario IDs
opus_sdd	v1	0/15	15	0	23.0	—
opus_sdd_a	v4	6/15	9	0	18.3	1, 2, 3, 4, 5, 6
opus_sdd_b	v4	5/15	10	0	20.5	1, 2, 3, 6, 15
opus_sdd_c	v4	6/15	9	0	18.2	1, 2, 3, 4, 5, 6
Reference						
oracle	v1	7/15	8	0	17.5	1, 2, 3, 4, 5, 11, 13
random_baseline	v1	0/15	15	0	23.0	—
target	v2	15/15	0	0	12.9	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15

Note. *Source: final_version_rows in the appendix data export.*

C.3 Strongest final version per model / workflow cell

For each of the four model × workflow cells, this section shows the per-scenario outcome of the strongest final version available in the data export, followed by the oracle reference controller. Where an intermediate version exceeds the final-version score (for example, the opus_sdd_a v1 = 9/15 result discussed in §7.3, while the final version records 6/15), that peak is intentionally discussed in the main body rather than duplicated here because C.3 is defined as a final-version comparison table. Intermediate-version peaks can be inspected directly in C.1 and results.csv.

GPT-5.4 PDD: gpt_pdd_group_a1 v3

Final version of gpt_pdd_group_a1: 4/15 scenarios solved, mean steps 19.5. Timeout step counts are omitted here; see Appendix C.1 for the per-scenario timeout cells.

ID	Scenario name	Outcome	Steps
S01	easy_open_short_path	**GOAL**	—
S02	easy_single_obstacle_detour	TIMEOUT	24
S03	easy_l_wall_one_turn	**GOAL**	—
S04	easy_corner_goal_blocked	TIMEOUT	26
S05	easy_two_isolated_obstacles	TIMEOUT	30
S06	medium_straight_narrow_corridor	**GOAL**	—
S07	medium_s_shaped_path	TIMEOUT	20

ID	Scenario name	Outcome	Steps
S08	medium_central_obstacle	TIMEOUT	20
S09	medium_single_valid_detour	TIMEOUT	18
S10	medium_dead_end_corridors	TIMEOUT	20
S11	hard_maze_one_dead_end	**GOAL**	—
S12	hard_u_trap_goal	TIMEOUT	24
S13	hard_long_detour_budget	TIMEOUT	26
S14	hard_two_dead_ends	TIMEOUT	28
S15	hard_move_away_corridor	TIMEOUT	22

GPT-5.4 SDD: gpt_sdd_b v4

Final version of gpt_sdd_b: 5/15 scenarios solved, mean steps 20.3. Timeout step counts are omitted here; see Appendix C.1 for the per-scenario timeout cells.

ID	Scenario name	Outcome	Steps
S01	easy_open_short_path	**GOAL**	—
S02	easy_single_obstacle_detour	**GOAL**	—
S03	easy_l_wall_one_turn	TIMEOUT	28
S04	easy_corner_goal_blocked	**GOAL**	—
S05	easy_two_isolated_obstacles	**GOAL**	—
S06	medium_straight_narrow_corridor	**GOAL**	—
S07	medium_s_shaped_path	TIMEOUT	20
S08	medium_central_obstacle	TIMEOUT	20
S09	medium_single_valid_detour	TIMEOUT	18
S10	medium_dead_end_corridors	TIMEOUT	20
S11	hard_maze_one_dead_end	TIMEOUT	24
S12	hard_u_trap_goal	TIMEOUT	24
S13	hard_long_detour_budget	TIMEOUT	26
S14	hard_two_dead_ends	TIMEOUT	28
S15	hard_move_away_corridor	TIMEOUT	22

Opus 4.7 PDD: —

No final-version Opus 4.7 PDD lineage records a success in this data subset; all three core lineages end at 0/15. See §7.4 of the main body.

Opus 4.7 SDD: opus_sdd_a v4

Final version of opus_sdd_a: 6/15 scenarios solved, mean steps 18.3. Timeout step counts are omitted here; see Appendix C.1 for the per-scenario timeout cells.

ID	Scenario name	Outcome	Steps
S01	easy_open_short_path	**GOAL**	—
S02	easy_single_obstacle_detour	**GOAL**	—
S03	easy_l_wall_one_turn	**GOAL**	—
S04	easy_corner_goal_blocked	**GOAL**	—
S05	easy_two_isolated_obstacles	**GOAL**	—
S06	medium_straight_narrow_corridor	**GOAL**	—
S07	medium_s_shaped_path	TIMEOUT	20
S08	medium_central_obstacle	TIMEOUT	20
S09	medium_single_valid_detour	TIMEOUT	18
S10	medium_dead_end_corridors	TIMEOUT	20
S11	hard_maze_one_dead_end	TIMEOUT	24
S12	hard_u_trap_goal	TIMEOUT	24
S13	hard_long_detour_budget	TIMEOUT	26
S14	hard_two_dead_ends	TIMEOUT	28
S15	hard_move_away_corridor	TIMEOUT	22

Reference: oracle v1

Final version of oracle: 7/15 scenarios solved, mean steps 17.5. Timeout step counts are omitted here; see Appendix C.1 for the per-scenario timeout cells.

ID	Scenario name	Outcome	Steps
S01	easy_open_short_path	**GOAL**	—
S02	easy_single_obstacle_detour	**GOAL**	—
S03	easy_l_wall_one_turn	**GOAL**	—
S04	easy_corner_goal_blocked	**GOAL**	—

ID	Scenario name	Outcome	Steps
S05	easy_two_isolated_obstacles	**GOAL**	—
S06	medium_straight_narrow_corridor	—	—
S07	medium_s_shaped_path	—	—
S08	medium_central_obstacle	—	—
S09	medium_single_valid_detour	—	—
S10	medium_dead_end_corridors	—	—
S11	hard_maze_one_dead_end	**GOAL**	—
S12	hard_u_trap_goal	—	—
S13	hard_long_detour_budget	**GOAL**	—
S14	hard_two_dead_ends	—	—
S15	hard_move_away_corridor	—	—

C.4 Pairwise drift decomposition

The table below reports the lineage-coarse drift decomposition for the four core model-by-workflow cells. Each non-initial version is classified by the repository drift analysis into exactly one of four outcomes relative to its immediate predecessor: same (behaviour preserved), pure drift (behaviour changed with neither net improvement nor regression), regression (success count reduced), or improvement (success count increased). The version-level classification is then expanded by 15 (one entry per scenario in the suite) to produce the n column below, so the table totals to 900 entries (60 non-initial core versions × 15 scenarios); the equivalent per-scenario slice of pairwise_changes.csv contains the same 900 classifications, and the full-repo per-scenario file contains 1215 rows when non-core approach folders are included. The two views are complementary: §7.6 gives the per-scenario pairwise decomposition used as the authoritative quantitative claim in the main body, while this appendix table gives a version-centric rollup that is easier to scan across lineages.

Cell	n	Same	Drift	Regression	Improvement	Drift rate
GPT-5.4 PDD core	315	204	46	12	53	0.352
GPT-5.4 SDD core	135	7	52	21	55	0.948
Opus 4.7 PDD core	315	87	162	29	37	0.724
Opus 4.7 SDD core	135	45	55	23	12	0.667
TOTAL	**900**	**343**	**315**	**85**	**157**	**0.619**

Drift rate = (drift + regression + improvement) / n. A drift rate of 1.0 means every adjacent scenario-level comparison showed a behavioural change of some kind. Source: pairwise_changes.csv in the submitted repository (core-subset filter applied).

C.5 Scenario suite reference

All 15 scenarios in the benchmark, with the difficulty tier, challenge type, grid dimensions, start and goal cells, and step budget. Every scenario ID used elsewhere in the appendix appears in this table.

ID	Name	Difficulty	Challenge type	Grid	Start	Goal	Budget
S01	easy_open_short_path	easy	straight line	5x3	(0, 1)	(4, 1)	20
S02	easy_single_obstacle_detour	easy	single detour	5x4	(0, 1)	(4, 1)	24
S03	easy_l_wall_one_turn	easy	single detour	6x5	(0, 0)	(5, 4)	28
S04	easy_corner_goal_blocked	easy	single detour	5x5	(0, 4)	(4, 0)	26
S05	easy_two_isolated_obstacles	easy	single detour	7x5	(0, 2)	(6, 2)	30
S06	medium_straight_narrow_corridor	medium	corridor	8x5	(0, 2)	(7, 2)	15
S07	medium_s_shaped_path	medium	corridor	6x5	(0, 0)	(5, 4)	20
S08	medium_central_obstacle	medium	single detour	7x7	(0, 3)	(6, 3)	20
S09	medium_single_valid_detour	medium	single detour	7x5	(0, 2)	(6, 2)	18
S10	medium_dead_end_corridors	medium	dead end	8x5	(0, 2)	(7, 2)	20
S11	hard_maze_one_dead_end	hard	dead end	8x6	(0, 0)	(7, 5)	24
S12	hard_u_trap_goal	hard	counterintuitive	7x7	(3, 6)	(3, 2)	24
S13	hard_long_detour_budget	hard	counterintuitive	10x7	(0, 3)	(9, 3)	26
S14	hard_two_dead_ends	hard	dead end	9x7	(0, 3)	(8, 6)	28
S15	hard_move_away_corridor	hard	counterintuitive	8x5	(5, 2)	(7, 2)	22

Note. *Source: scenario_metadata_rows in the appendix data export.*

C.6 Reproducibility

All artefacts required to produce the tables shown in this appendix are present in the provided repository. The steps involved in reproducing the data are as follows: (1) `pip install -e .`: The command will read the pyproject.toml file and install gridbot alongside all its dependencies; (2) `python run_experiment.py`: This will execute the full benchmark by calling gridbot/eval/benchmark_suite.py with each controller present in the controllers/ directory, resulting in the production of results.csv and other summary CSV files; (3) `python analyse_traces.py` (alternatively, `python analyse_results`): This will re-generate chain_summary.csv, pairwise_changes.csv and pairwise_summary.csv from results.csv; (4) `python tools/build_appendix_tables_data.py`: This will re-generate the JSON file used to build the tables in this appendix, while `node tools/build_appendix_tables_workbook.mjs` will rebuild the xlsx workbook; (5) `node tools/verify_appendix_tables.mjs`: This will verify the reconstructed appendix workbook and generate preview images from the source CSV files. The test suite is executed by `pytest tests -q` and will

succeed in 15 tests. It is expected that running the benchmark in (2) on a modern laptop takes approximately three minutes.

Data-source mapping. results.csv is the master per-run record and is the source for every aggregate in this appendix. chain_summary.csv provides per-lineage aggregates and is the source for C.2. pairwise_changes.csv provides the per-scenario adjacent-version drift classifications (1215 rows for the full repo) and is the authoritative source for the drift decomposition in §7.6; the core-subset view in C.4 is a filtered aggregate of the same file. pairwise_summary.csv is the rolled-up version-pair summary (81 rows) and is useful for lineage-level rollups. The benchmark scenarios are defined in gridbot/eval/benchmark_suite.py and are the source for C.5. The submission commit hash is 132e856c785a66938bf2f35ee32d1c9968076f52.

Reproducibility checklist. The following items are present and verifiable in the submitted repository at commit 132e856c785a66938bf2f35ee32d1c9968076f52:

- Simulator source, scenario suite, and controller-loader (gridbot/).
- All 107 versioned controller source files under approaches/, organised by lineage.
- Raw per-run output (results.csv, 1605 rows) plus the three derived CSVs (chain_summary.csv, pairwise_changes.csv, pairwise_summary.csv).
- Reproduction scripts: run_experiment.py, analyse_traces.py, analyse_results, and tools/build_appendix_tables_data.py / tools/build_appendix_tables_workbook.mjs / tools/verify_appendix_tables.mjs.
- Project configuration in pyproject.toml (installable via pip install -e .).
- Test suite in tests/ (15 tests; pytest tests -q from the project root).
- Prompt artefacts for GPT-5.4 and Opus 4.7 PDD and SDD lineages (excerpts in Appendix A; full files in the repository under prompts/).

Two items are intentionally out of scope of this artefact: (i) bit-level reproduction of the generated controllers from their prompts, for the reasons given in §5.2 (consumer chat interface, sampling parameters not controllable); and (ii) extensions of the benchmark beyond the 15 scenarios defined in gridbot/eval/benchmark_suite.py.